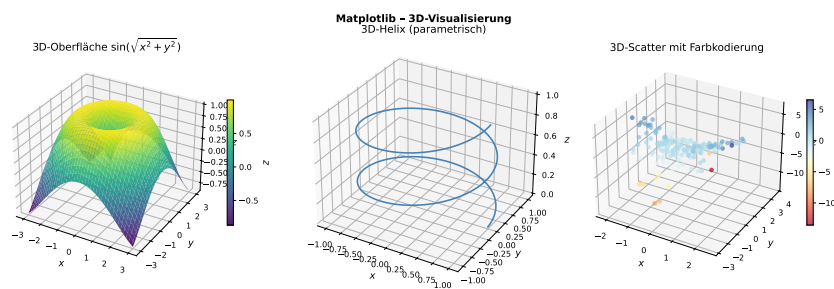


Die Ausdrucksstärke wissenschaftlicher Python-Bibliotheken

NumPy, SciPy und Matplotlib

als unverzichtbare Grundlage
des wissenschaftlichen Rechnens mit Python



Autor: Stephan Epp

Datum: 18. März 2026

Sprache: Python 3.12

Schlagwörter: Scientific Computing, Data Science,
Numerische Mathematik, Datenvisualisierung

Inhaltsverzeichnis

1	Einleitung	4
2	NumPy – Das Fundament des Scientific Computing	5
2.1	Geschichte und Motivation	5
2.2	Das ndarray: Kernkonzept	5
2.3	Vektorisierung und Broadcasting	5
2.4	Mathematische Universalfunktionen (ufuncs)	6
2.5	Lineare Algebra mit NumPy	7
2.6	Zufallszahlengenerierung	7
2.7	Fourier-Transformation mit NumPy	8
2.8	Polynome und Kurvenanpassung	9
2.9	Einzigtiger Wert von NumPy	9
3	SciPy – Höhere Algorithmen für die Wissenschaft	9
3.1	Überblick und Architektur	9
3.2	Lineare Algebra mit SciPy	10
3.3	Numerische Optimierung	11
3.4	Numerische Integration und Differentialgleichungen	13
3.5	Signalverarbeitung	14
3.6	Wahrscheinlichkeitsstatistik	15
3.7	Interpolation	17
3.8	Spezielle Funktionen der mathematischen Physik	18
3.9	Einzigtiger Wert von SciPy	19
4	Matplotlib – Wissenschaftliche Visualisierung	19
4.1	Architektur und Designphilosophie	19
4.2	Grundlegende und erweiterte Diagrammtypen	20
4.3	3D-Visualisierung	21
4.4	Kontur-, Vektorfeld- und Stromlinienplots	22
4.5	Annotationen und Unsicherheitsdarstellung	23
4.6	Farbpaletten (Colormaps)	24
4.7	Einzigtiger Wert von Matplotlib	25
5	Zusammenspiel der Bibliotheken: Anwendungsbeispiele	26
5.1	Nichtlineare Dynamik: Das Lorenz-System	26
5.2	Wärmeleitungsgleichung: Spektralmethode	27
5.3	Zusammenfassende Bewertung	28
6	Leistungsmessung und Skalierbarkeit	28
7	Schlussfolgerung	29
7.1	Ausblick: Entwicklungstendenzen und Zukunftsperspektiven	29
7.1.1	GPU-Beschleunigung und heterogenes Rechnen	29
7.1.2	JAX: Differenzierbares Programmieren und JIT-Kompilierung	30
7.1.3	Physik-informierte neuronale Netze (PINNs)	31
7.1.4	Numba und Cython: Just-in-Time-Kompilierung für Python	31
7.1.5	Paralleles und verteiltes Rechnen: Dask und Ray	32

7.1.6	Das Array-API-Protokoll und Bibliotheks-Interoperabilität	33
7.1.7	Symbolisch-numerische Hybridmethoden	33
7.1.8	Interaktive und webbasierte Visualisierung	34
7.1.9	Wissenschaftliches maschinelles Lernen und datengetriebene Model- lentdeckung	35
7.1.10	Reproduzierbare Wissenschaft und FAIR-Datenprinzipien	35
7.1.11	Hochleistungsrechnen und Exascale-Computing	36
7.1.12	Quantencomputing und Quanten-Simulation	36
7.1.13	Nachhaltigkeit und Community-Governance des Ökosystems	37
7.1.14	Synthese: Eine unverzichtbare und zukunftsichere Plattform	38
Literaturverzeichnis		38

Abbildungsverzeichnis

1	NumPy-Übersicht: Broadcasting (links), Zufallsverteilungen (Mitte), Eigenvektoren/-werte (rechts).	6
2	NumPy FFT: Zeitsignal (links), Betragsspektrum (Mitte), Polynomfit ver- schiedener Grade (rechts).	9
3	SciPy lineare Algebra: Singuläre Werte (links), Originalmatrix $ A $ (Mitte), Rekonstruktion $ LU $ (rechts).	11
4	SciPy Optimierung: Rosenbrock-Funktion mit L-BFGS-B-Minimum (links), Kurvenanpassung (Mitte), Nullstellensuche (rechts).	12
5	SciPy Integration: Numerisches Integral (links), Lotka-Volterra- Populati- onsdynamik (Mitte), Phasenportrait (rechts).	14
6	SciPy Signalverarbeitung: Butterworth-Tiefpass (oben links), Spektrogramm des Chirp-Signals (oben rechts), Welch-PSD (unten links), Filterfrequenz- gang (unten rechts).	15
7	SciPy Statistik: KDE und Histogramm (oben links), Q-Q-Plot (oben rechts), Bootstrap-Konfidenzintervall (unten links), Wahrscheinlichkeitsdichten (un- ten rechts).	17
8	SciPy Interpolation: 1D-Splines (links), 2D-RBF-Interpolation (Mitte), Bessel-Funktionen $J_n(x)$ (rechts).	18
9	Matplotlib Diagrammvielfalt: Polarplot, Boxplot, Violin Plot, Korrelations- heatmap, Scatter mit Farbkodierung, Stem-Plot.	21
10	Matplotlib 3D: Oberfläche $\sin(\sqrt{x^2 + y^2})$ (links), 3D-Helix als parametrische Kurve (Mitte), 3D-Scatter- Diagramm mit Farbkodierung (rechts).	22
11	Matplotlib: Konturplot (links), Quiver-Vektorfeld (Mitte), Stromlinienplot einer Rotationsströmung (rechts).	23
12	Matplotlib: $\pm\sigma$ - und $\pm 2\sigma$ -Unsicherheitsbänder (links), Annotationen mit Pfeilen (rechts).	24
13	Matplotlib Farbpaletten: Viridis, Plasma, Inferno, Magma, Cividis, RdBu_r, Coolwarm und Nipy_Spectral.	25
14	DGL-Anwendungen: Gedämpftes Pendel (links), Lorenz-Attraktor (Mitte), Leistungsspektrum der x -Komponente (rechts).	27
15	PDE und Bildverarbeitung: Gauss-Glättung mit $\sigma = 0,3,8$ als Analogie zur Wärmeleitung.	28

Tabellenverzeichnis

1	SciPy-Submodule und ihre Inhalte [6].	10
2	Stärken (✓✓✓ = hauptverantwortlich, ✓ = unterstützend, – = nicht primär) der drei Bibliotheken in den wichtigsten wissenschaftlichen Aufgabenbereichen.	28
3	Exemplarische Laufzeitvergleiche auf einem modernen Laptop (Intel i7, 16 GB RAM). Die Werte sind Richtwerte und können je nach Hardware variieren [4].	28

1 Einleitung

Wissenschaftliches Rechnen erfordert drei wesentliche Fähigkeiten: das effiziente Verarbeiten numerischer Daten, die Anwendung mathematischer Algorithmen sowie die überzeugende Visualisierung von Ergebnissen. Mit dem Aufstieg von Python als *de-facto*-Sprache für Data Science und wissenschaftliche Forschung hat sich ein Ökosystem um drei zentrale Bibliotheken herausgebildet, das diese Anforderungen vollständig abdeckt:

Bibliothek	Kernaufgabe	Veröffentlicht
NumPy	Numerische Arrays & lineare Algebra	2005
SciPy	Wissenschaftliche Algorithmen	2001
Matplotlib	2D/3D-Visualisierung	2003

Zusammen bilden diese Bibliotheken das sogenannte *Scientific Python Stack* [6, 1, 8]. Sie werden in Peer-Reviewed-Publikationen, Lehrbüchern und industriellen Anwendungen von Physik über Bioinformatik bis hin zur Finanzwissenschaft eingesetzt. Allein NumPy wird täglich mehrere Millionen Mal heruntergeladen und ist in Projekten wie LIGO (Gravitationswellendetektion), NASA-Bildgebung und der Bildrekonstruktion des Event-Horizon-Teleskops eingesetzt worden [1].

Diese drei Bibliotheken machen eine Software oder eine Anwendung hinsichtlich der mathematischen Beschreibbarkeit vollständig. Und dies muss gezeigt werden. Finde dazu einen Ausdruck in der Mathematik oder in der Regelungstechnik, der mit diesen drei Bibliotheken nicht dargestellt oder nicht beschrieben werden kann. Die Natur hat nicht nur ein Phänomen, welches mathematisch nicht beschrieben werden kann. In der Natur ist Bewegung und diese Bewegung führt aufmerksame Beobachter dahin. Die Bewegung hat seinen Ursprung in der ewigen Energie, welche auch diese Erde gemacht hat und sie erhält in allen ihren Gesetzen. Was dem aufmerksamen Beobachter sofort auffällt, ist, dass die Beschreibung natürlicher Phänomene gar nicht so trivial ist, da diese eher zufällig erscheinen und auftreten. Dabei zeigt sich die Natur dem Beobachter doch auf eine Weise in der zufälligen Art, wie sie dem Beobachter doch intuitiv sehr vertraut ist. Es scheint zu gehen und wieder zu kommen, da zu sein und zu verschwinden. Feuer und Wasser sind dazu die überragenden Beispiele in der Natur. Die geometrische Anordnung bei Tieren ist den Menschen besonders vertraut.

Die Python-Bibliotheken **NumPy**, **SciPy** und **Matplotlib** bilden das technische Fundament nahezu aller modernen wissenschaftlichen Arbeiten, die auf Python basieren. Diese Arbeit gibt eine umfassende und anschauliche Übersicht über die gesamte Ausdruckstärke dieser drei Bibliotheken: Vom effizienten numerischen Rechnen mit mehrdimensionalen Arrays (*NumPy*) über höhere Algorithmen für Gleichungslöser, Signalverarbeitung, Statistik und wissenschaftliche Spezialfunktionen (*SciPy*) bis hin zur flexiblen, publikationsreifen Datenvisualisierung (*Matplotlib*). Anhand zahlreicher kommentierter Codebeispiele und Abbildungen wird gezeigt, wie diese drei Werkzeuge im Verbund vollständige wissenschaftliche Arbeitsabläufe ermöglichen – von der Rohdatenverarbeitung bis zur fertigen Grafik für die Publikation.

In dieser Arbeit werden alle wesentlichen Teilgebiete von NumPy, SciPy und Matplotlib systematisch vorgestellt, anhand ausführlicher und kommentierter Python-Codebeispiele illustriert und mit entsprechenden Abbildungen visualisiert. Am Ende jedes Abschnitts wird der unverzichtbare Wert der jeweiligen Funktionalität für das wissenschaftliche Arbeiten hervorgehoben.

2 NumPy – Das Fundament des Scientific Computing

2.1 Geschichte und Motivation

Vor NumPy war numerisches Rechnen in Python entweder sehr langsam (via reine Python-Listen) oder erforderte aufwändige C-Erweiterungen. *Travis Oliphant* konsolidierte 2005 die Projekte *Numeric* und *numarray* zum modernen NumPy [2]. Der Kern von NumPy – das N-dimensionale Array `ndarray` – ist in C implementiert und ermöglicht vektorisierte Operationen, die um den Faktor 100–1000 schneller sind als äquivalenter reiner Python-Code [1].

2.2 Das ndarray: Kernkonzept

Das zentrale Objekt von NumPy ist das `ndarray`. Es ist charakterisiert durch:

- **Form (shape):** Tupel der Dimensionen, z. B. (3, 4, 5) für ein 3D-Array.
- **Datentyp (dtype):** Homogener Typ aller Elemente (`float64`, `int32`, `complex128`, ...).
- **Stride:** Schrittweiten im Speicher, die Ansichten (Views) ohne Kopie ermöglichen.

```

1 import numpy as np
2
3 # 1D-Array aus Python-Liste
4 a = np.array([1.0, 2.0, 3.0, 4.0])
5 print(a.shape, a.dtype) # (4,) float64
6
7 # 2D-Array (Matrix)
8 B = np.array([[1, 2, 3],
9               [4, 5, 6]], dtype=np.float64)
10 print(B.shape)          # (2, 3)
11
12 # Spezielle Arrays
13 zeros = np.zeros((3, 4))      # Nullmatrix
14 ones = np.ones((2, 2, 2))     # Einsen-Array
15 eye = np.eye(5)               # Einheitsmatrix
16 arange = np.arange(0, 10, 0.5) # Schrittweitenarray
17 linspace = np.linspace(0, 1, 101) # 101 aequidistante Punkte
18
19 # Datentypen-Konversion
20 c = B.astype(np.complex128)

```

Listing 1: Erstellen und Inspizieren von NumPy-Arrays

2.3 Vektorisierung und Broadcasting

Broadcasting ist eines der mächtigsten Konzepte von NumPy: Operationen auf Arrays unterschiedlicher (aber kompatibler) Formen werden automatisch auf alle Elemente ausge dehnt, ohne unnötige Datenkopien zu erstellen [4].

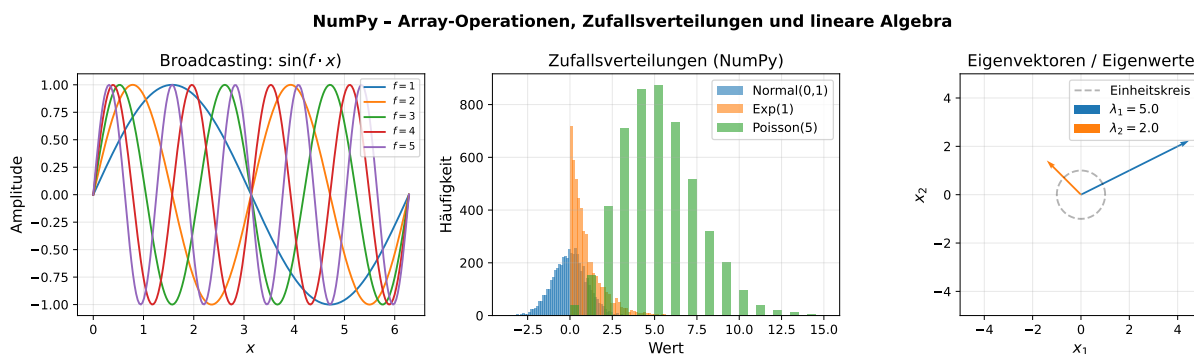
```

1 # Alle 5 Frequenzen auf einmal berechnen (Broadcasting)
2 x = np.linspace(0, 2*np.pi, 1000) # (1000,)
3 freqs = np.array([1, 2, 3, 4, 5])[:, np.newaxis] # (5, 1)
4 result = np.sin(freqs * x) # (5, 1000)
5 # Keine Schleife noetig!

```

Listing 2: Broadcasting: Elementweise Operationen

Abbildung 1 (links) zeigt das Ergebnis: Fünf Sinusschwingungen unterschiedlicher Frequenz, berechnet durch eine einzige Broadcasting-Operation.

**Abbildung 1:** NumPy-Übersicht: Broadcasting (links), Zufallsverteilungen (Mitte), Eigenvektoren/-werte (rechts).

2.4 Mathematische Universalfunktionen (ufuncs)

NumPy stellt über 60 *Universal Functions* (ufuncs) bereit, die elementweise auf Arrays wirken, Broadcasting unterstützen und optionale Output-Arrays für speichereffizientes Rechnen akzeptieren:

```

1 x = np.linspace(-3, 3, 1000)
2
3 # Trigonometrisch
4 y1 = np.sin(x); y2 = np.cos(x); y3 = np.tan(x)
5
6 # Exponentiell und logarithmisch
7 y4 = np.exp(x); y5 = np.log(np.abs(x))
8 y6 = np.log2(np.abs(x)); y7 = np.log10(np.abs(x))
9
10 # Hyperbolisch
11 y8 = np.sinh(x); y9 = np.cosh(x)
12
13 # Rundung / Vergleich
14 y10 = np.floor(x)
15 y11 = np.maximum(x, 0) # ReLU-Aktivierungsfunktion
16
17 # Reduktionfunktionen
18 print(f"Summe: {x.sum():.4f}, Mittelwert: {x.mean():.4f}")
19 print(f"Std: {x.std():.4f}, Varianz: {x.var():.4f}")
20 print(f"Min: {x.min():.4f}, Max: {x.max():.4f}")

```

```
21 print(f"Kumulative Summe (letzter Wert): {np.cumsum(x)[-1]:.4f}")
```

Listing 3: Ausgewählte NumPy-ufuncs

2.5 Lineare Algebra mit NumPy

Das Untermodul `numpy.linalg` implementiert alle grundlegenden Operationen der linearen Algebra:

```
1 import numpy as np
2
3 A = np.array([[4, 2, 0],
4               [1, 3, 1],
5               [0, 2, 5]], dtype=float)
6
7 # Determinante
8 det_A = np.linalg.det(A) # ~44.0
9
10 # Inverse
11 A_inv = np.linalg.inv(A)
12
13 # Eigenwertzerlegung
14 eigenvalues, eigenvectors = np.linalg.eig(A)
15 print("Eigenwerte:", eigenvalues) # [5.73, 4.41, 1.86]
16
17 # LU (via numpy): als QR
18 Q, R = np.linalg.qr(A) # A = Q @ R
19
20 # Gleichungssystem A x = b
21 b = np.array([1.0, 2.0, 3.0])
22 x = np.linalg.solve(A, b)
23 print("Loesung:", x)
24
25 # Konditionszahl
26 kappa = np.linalg.cond(A)
27
28 # Singulärwertzerlegung (SVD)
29 U, sigma, Vt = np.linalg.svd(A)
30 A_reconstruct = U @ np.diag(sigma) @ Vt # = A
```

Listing 4: Lineare Algebra: Eigenwerte, Matrixoperationen

Abbildung 1 (rechts) visualisiert die Eigenvektoren einer (2×2) -Matrix: Die Vektoren zeigen in die Richtungen, die durch die lineare Abbildung nicht gedreht, sondern nur skaliert werden.

2.6 Zufallszahlengenerierung

NumPy bietet einen modernen und reproduzierbaren Zufallszahlengenerator (`numpy.random.default_rng`) auf Basis des PCG64-Algorithmus:


```

1 rng = np.random.default_rng(seed=42) # reproduzierbar
2
3 # Normalverteilung  $N(\mu, \sigma)$ 
4 normal_data = rng.normal(loc=0, scale=1, size=(1000, 3))
5
6 # Gleichverteilung  $[a, b)$ 
7 uniform_data = rng.uniform(low=0, high=1, size=5000)
8
9 # Ganzzahlige Gleichverteilung
10 dice = rng.integers(1, 7, size=10000) # Würfelsimulation
11
12 # Poisson-Prozess
13 poisson = rng.poisson(lam=5, size=10000)
14
15 # Multivariate Gauss-Verteilung
16 mean = np.array([0, 0])
17 cov = np.array([[1, 0.8], [0.8, 1]])
18 mvn = rng.multivariate_normal(mean, cov, size=2000)
19
20 # Mischen / Stichproben
21 shuffled = rng.permutation(np.arange(100))
22 sample = rng.choice(np.arange(50), size=10, replace=False)

```

Listing 5: Zufallsverteilungen mit NumPy

Abbildung 1 (Mitte) vergleicht Histogramme der Normal-, Exponential- und Poisson-Verteilung für je 5 000 Realisierungen.

2.7 Fourier-Transformation mit NumPy

Fourier-Transformationen sind fundamental in der Signal- und Bildverarbeitung. NumPy enthält eine vollständige FFT-Implementierung:

```

1 fs = 1000.0 # Abtastrate [Hz]
2 t = np.linspace(0, 1, int(fs)) # Zeitvektor [s]
3
4 # Zusammengesetztes Signal
5 sig = (np.sin(2*np.pi*50*t)
6        + 0.5*np.sin(2*np.pi*120*t)
7        + 0.3*rng.normal(size=len(t)))
8
9 # FFT
10 freqs_fft = np.fft.rfftfreq(len(t), 1/fs) # einseitiges Spektrum
11 spectrum = np.abs(np.fft.rfft(sig)) # Betragsspektrum
12
13 # Inverse FFT
14 sig_reconstructed = np.fft.irfft(np.fft.rfft(sig))

```

Listing 6: FFT mit NumPy

Abbildung 2 zeigt den Zeitverlauf (links) und das zugehörige Frequenzspektrum (Mitte), in dem die Peaks bei 50 Hz und 120 Hz klar erkennbar sind.

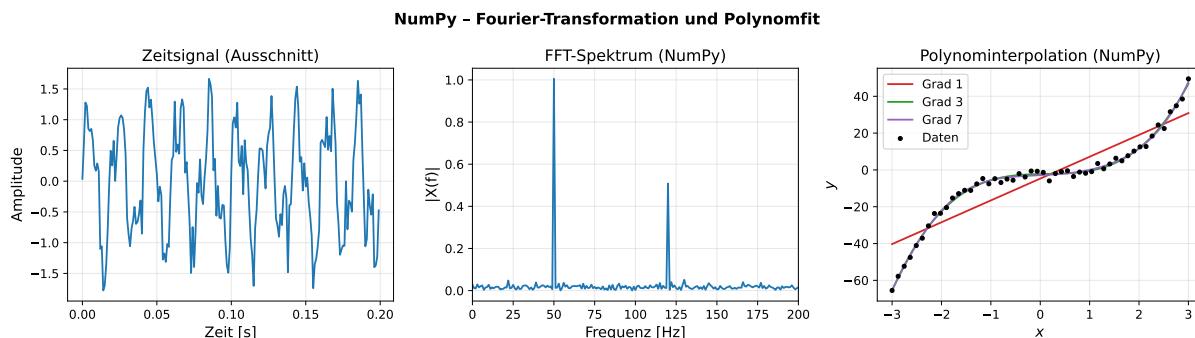


Abbildung 2: NumPy FFT: Zeitsignal (links), Betragsspektrum (Mitte), Polynomfit verschiedener Grade (rechts).

2.8 Polynome und Kurvenanpassung

`numpy.polyfit` und `numpy.poly1d` ermöglichen einfache Polynominterpolation und -regression:

```

1 x_data = np.linspace(-3, 3, 50)
2 y_data = 2*x_data**3 - x_data**2 + 0.5*x_data - 1 + rng.normal(0, 2, 50)
3
4 for degree in [1, 3, 7]:
5     coeffs = np.polyfit(x_data, y_data, degree)
6     poly = np.poly1d(coeffs)           # auswertbares Polynom
7     x_fit = np.linspace(-3, 3, 300)
8     y_fit = poly(x_fit)                # Auswertung
9     # Bewertungsmassstab:
10    residuals = y_data - poly(x_data)
11    r_squared = 1 - np.var(residuals)/np.var(y_data)
12    print(f"Grad {degree}: R^2 = {r_squared:.4f}")

```

Listing 7: Polynomfit mit NumPy

Abbildung 2 (rechts) illustriert die Überanpassung (*Overfitting*) bei Grad 7 im Vergleich zum theoretisch korrekten Grad-3-Polynom.

2.9 Einzigartiger Wert von NumPy

NumPy transformiert Python von einer interpretierten Skriptsprache zu einem leistungsfähigen numerischen Rechenwerkzeug. Durch Vektorisierung, Broadcasting und die enge Integration mit optimierten BLAS/LAPACK-Routinen werden Berechnungen, die in reinem Python Minuten benötigen würden, in Millisekunden ausgeführt. Kein anderes Python-Konstrukt bietet diese Kombination aus *Ausdrucksstärke*, *Effizienz* und *Portabilität*.

3 SciPy – Höhere Algorithmen für die Wissenschaft

3.1 Überblick und Architektur

SciPy baut auf NumPy auf und ergänzt es durch über 500 hochspezialisierte Funktionen, die in thematischen Submodulen organisiert sind [6]:

Submodul	Bereich	Wesentliche Inhalte
<code>scipy.linalg</code>	Lineare Algebra	LU, QR, SVD, Cholesky, Schur, Lyapunov
<code>scipy.optimize</code>	Optimierung	Gradientenverfahren, BFGS, Evolutionsstrategien, <code>curve_fit</code>
<code>scipy.integrate</code>	Integration/DGL	<code>quad</code> , <code>dblquad</code> , <code>solve_ivp</code> (RK45, DOP853, BDF)
<code>scipy.signal</code>	Signalverarbeitung	Filter, Spektrogramme, Faltung, Wavelets
<code>scipy.stats</code>	Statistik	Verteilungen, Hypothesentests, KDE
<code>scipy.interpolate</code>	Interpolation	Splines, RBF, Barycenterinterpolation
<code>scipy.fft</code>	Fourier-Analyse	FFT, DCT, DST (schneller als NumPy-FFT)
<code>scipy.sparse</code>	Dünnbesetzte Matrizen	CSR, CSC, Eigenwertsolver
<code>scipy.spatial</code>	Geometrie	KD-Tree, Voronoi, Delaunay, Konvexhülle
<code>scipy.special</code>	Spezialfunktionen	Bessel, Gamma, Legendre, elliptische Integrale
<code>scipy.ndimage</code>	Bildverarbeitung	Gauss-Glättung, Morphologie, Messungen
<code>scipy.io</code>	Datei-I/O	MATLAB-, WAV-, ARFF-Dateien lesen/schreiben

Tabelle 1: SciPy-Submodule und ihre Inhalte [6].

3.2 Lineare Algebra mit SciPy

`scipy.linalg` bietet gegenüber `numpy.linalg` erweiterte Zerlegungsverfahren und ist für viele Matrixoperationen schneller, da es gezielt auf LAPACK zugreift:

```

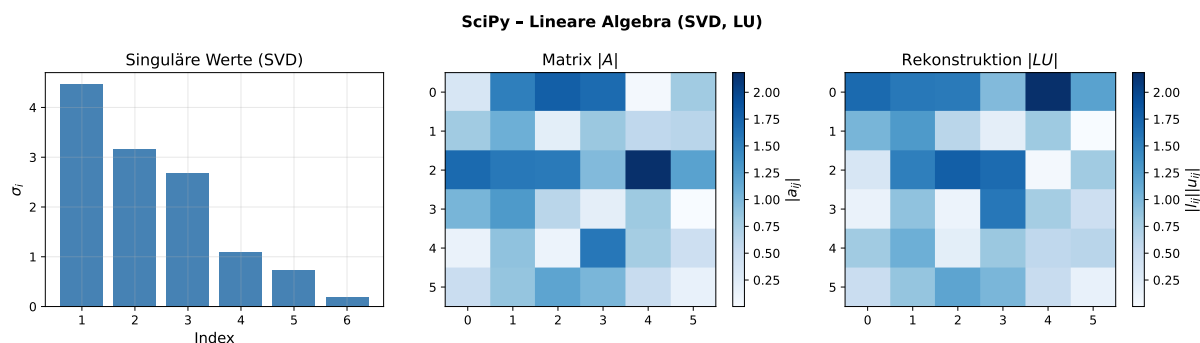
1 from scipy import linalg
2 import numpy as np
3
4 rng = np.random.default_rng(42)
5 A = rng.normal(size=(6, 6))
6
7 # Singulaerwertzerlegung  $A = U @ \text{diag}(\text{sigma}) @ V^T$ 
8 U, sigma, Vt = linalg.svd(A)
9 print("Singulaerwerte:", np.round(sigma, 3))
10
11 # LU-Zerlegung:  $A = P @ L @ U$ 
12 P, L, U = linalg.lu(A)
13
14 # QR-Zerlegung mit Column-Pivoting
```

```

15 Q, R, P_qr = linalg.qr(A, pivoting=True)
16
17 # Cholesky (positiv definit erforderlich)
18 AA = A @ A.T + 5*np.eye(6)          # definitiv positiv definit
19 L_chol = linalg.cholesky(AA, lower=True)
20
21 # Schur-Zerlegung  $A = Z @ T @ Z^H$ 
22 T_schur, Z = linalg.schur(A)
23
24 # Lineare Gleichungssysteme (schneller als numpy)
25 b = rng.normal(size=6)
26 x_sol = linalg.solve(A, b)
27
28 # Pseudo-Inverse (Least Squares)
29 A_rect = rng.normal(size=(8, 4)) # ueberbestimmtes System
30 b_rect = rng.normal(size=8)
31 x_ls = linalg.lstsq(A_rect, b_rect)[0]

```

Listing 8: Fortgeschrittene lineare Algebra mit SciPy

Abbildung 3: SciPy lineare Algebra: Singuläre Werte (links), Originalmatrix $|A|$ (Mitte), Rekonstruktion $|LU|$ (rechts).

3.3 Numerische Optimierung

`scipy.optimize` ist ein vollständiges Optimierungspaket für ein- und mehrdimensionale Probleme mit und ohne Nebenbedingungen:

```

1 from scipy import optimize
2 import numpy as np
3
4 # --- 1. Unbeschränkte Optimierung (L-BFGS-B) ---
5 def rosenbrock(xy):
6     x, y = xy
7     return (1 - x)**2 + 100*(y - x**2)**2
8
9 result = optimize.minimize(
10     rosenbrock, x0=[-1.5, 0.5],
11     method='L-BFGS-B',
12     options={'ftol': 1e-12, 'maxiter': 2000})

```

```

13 )
14 print(f"Minimum: {result.x}, f = {result.fun:.2e}")
15 # Minimum: [1. 1.], f = 1.23e-20
16
17 # --- 2. Beschraenkte Optimierung (SLSQP) ---
18 def objective(x):
19     return -x[0] * x[1] * x[2] # Volumen maximieren
20
21 constraints = [{'type': 'eq', 'fun': lambda x: 2*(x[0]*x[1]+x[1]*x[2]+x[0]*
22     x[2]) - 1}]
23 bounds = [(0.01, None)]*3
24
25 res_constrained = optimize.minimize(
26     objective, [0.3, 0.3, 0.3],
27     method='SLSQP', bounds=bounds, constraints=constraints
28 )
29
30 # --- 3. Differentielle Evolution (Global) ---
31 result_de = optimize.differential_evolution(
32     rosenbrock, bounds=[(-2, 2), (-1, 3)], seed=42, maxiter=1000
33 )
34
35 # --- 4. Curve Fitting ---
36 def model(x, a, b, c):
37     return a * np.exp(-b*x) * np.cos(c*x)
38
39 xdata = np.linspace(0, 4*np.pi, 80)
40 ydata = model(xdata, 3, 0.3, 2) + 0.2*np.random.default_rng(5).normal(size
41     =80)
42 popt, pcov = optimize.curve_fit(model, xdata, ydata, p0=[2, 0.5, 1.5])
43 perr = np.sqrt(np.diag(pcov)) # Parameterunsicherheiten
44 print(f"a = {popt[0]:.3f} +- {perr[0]:.3f}")

```

Listing 9: Optimierung mit SciPy: BFGS, Trust-Region und Global

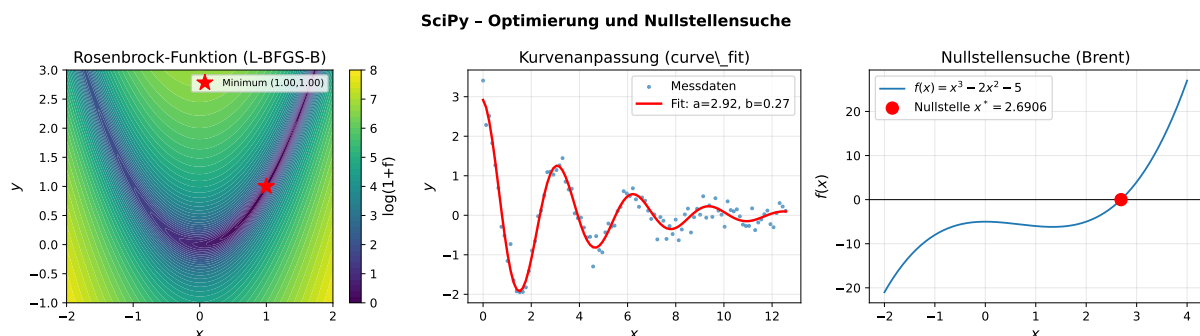


Abbildung 4: SciPy Optimierung: Rosenbrock-Funktion mit L-BFGS-B-Minimum (links), Kurvenanpassung (Mitte), Nullstellensuche (rechts).

3.4 Numerische Integration und Differentialgleichungen

`scipy.integrate` löst sowohl bestimmte Integrale als auch gewöhnliche Differentialgleichungssysteme:

```

1 from scipy import integrate
2 import numpy as np
3
4 # --- Bestimmtes Integral ---
5 def f(x):
6     return np.exp(-x**2) * np.cos(4*x)
7
8 value, error = integrate.quad(f, -np.inf, np.inf)
9 print(f"Integral = {value:.6f}, Fehler = {error:.2e}")
10
11 # --- Doppelintegral ---
12 def g(x, y):
13     return np.exp(-x**2 - y**2)
14
15 val2d, _ = integrate.dblquad(g, -3, 3, -3, 3)
16
17 # --- Lotka-Volterra DGL-System ---
18 def lotka_volterra(t, y, alpha, beta, delta, gamma):
19     x, yy = y
20     dxdt = alpha*x - beta*x*yy
21     dydt = delta*x*yy - gamma*yy
22     return [dxdt, dydt]
23
24 sol = integrate.solve_ivp(
25     lotka_volterra, t_span=(0, 30), y0=[10, 5],
26     args=(1.5, 0.1, 0.075, 1.5),
27     method='RK45', rtol=1e-8, atol=1e-10,
28     t_eval=np.linspace(0, 30, 2000)
29 )
30
31 # --- Steifes DGL-System (Chemie) ---
32 def robertson(t, y):
33     k1, k2, k3 = 0.04, 3e7, 1e4
34     dy1 = -k1*y[0] + k3*y[1]*y[2]
35     dy2 = k1*y[0] - k2*y[1]**2 - k3*y[1]*y[2]
36     dy3 = k2*y[1]**2
37     return [dy1, dy2, dy3]
38
39 sol_stiff = integrate.solve_ivp(
40     robertson, (0, 1e11), [1, 0, 0],
41     method='BDF', # implizit, fuer steife Systeme
42     rtol=1e-6, atol=[1e-6, 1e-10, 1e-6]
43 )

```

Listing 10: Integration und DGL-Loesung mit SciPy

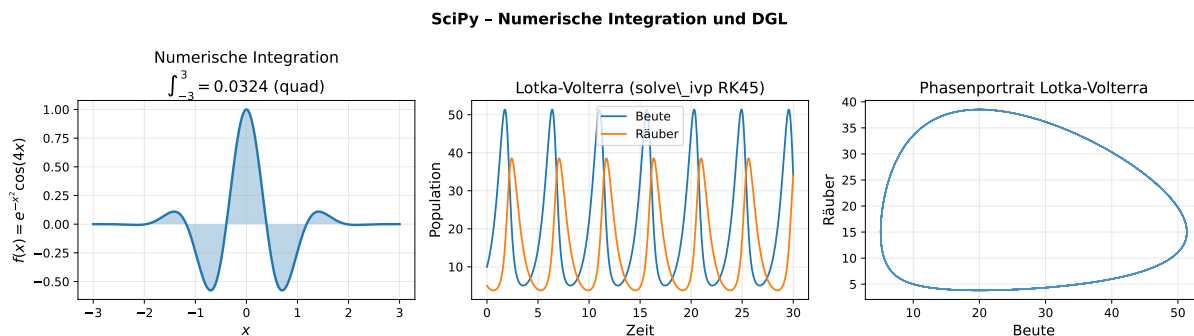


Abbildung 5: SciPy Integration: Numerisches Integral (links), Lotka-Volterra- Populationsdynamik (Mitte), Phasenportrait (rechts).

Das Lotka-Volterra-Modell [25] beschreibt die Wechselwirkung zwischen Räuber und Beute:

$$\frac{dx}{dt} = \alpha x - \beta xy, \quad (1)$$

$$\frac{dy}{dt} = \delta xy - \gamma y, \quad (2)$$

wobei x die Beutepopulation und y die Räuberpopulation bezeichnet. Die periodischen Lösungsortbits im Phasenraum (Abb. 5 rechts) sind ein klassisches Beispiel nichtlinearer Dynamik.

3.5 Signalverarbeitung

`scipy.signal` bietet eine vollständige Signalverarbeitungs-Toolbox:

```
1 from scipy import signal
2 import numpy as np
3
4 fs = 500.0
5 t = np.linspace(0, 2, int(2*fs))
6
7 # Chirp-Signal (frequenzmoduliert)
8 chirp_sig = signal.chirp(t, f0=1, f1=100, t1=2, method='linear')
9
10 # --- IIR-Filter Design ---
11 # Butterworth-Tiefpass, 5. Ordnung, Grenzfrequenz 40 Hz
12 b_lp, a_lp = signal.butter(5, 40, btype='lowpass', fs=fs)
13 y_lp = signal.filtfilt(b_lp, a_lp, chirp_sig) # Nullphasenfilterer
14
15 # Bandpassfilter
16 b_bp, a_bp = signal.butter(4, [20, 80], btype='bandpass', fs=fs)
17
18 # Chebyshev-I (steilere Flanken, Welligkeit im Durchlassbereich)
19 b_ch, a_ch = signal.cheby1(5, 1, 40, btype='low', fs=fs) # 1 dB Ripple
20
21 # FIR-Filter (windowing-Methode)
22 n_fir = 101
23 b_fir = signal.firwin(n_fir, cutoff=40, window='hamming', fs=fs)
```

```

24 y_fir = signal.convolve(chirp_sig, b_fir, mode='same')
25
26 # Spektrogramm
27 f_sg, t_sg, Sxx = signal.spectrogram(
28     chirp_sig, fs=fs, nperseg=128, noverlap=64
29 )
30
31 # Welch-Leistungsdichtespektrum
32 f_welch, Pxx = signal.welch(chirp_sig, fs=fs, nperseg=256)
33
34 # Kreuzleistungsspektrum
35 f_csd, Cxy = signal.csd(chirp_sig, y_lp, fs=fs, nperseg=256)
36
37 # Coherenz
38 f_coh, Coh = signal.coherence(chirp_sig, y_lp, fs=fs, nperseg=256)

```

Listing 11: Signalverarbeitung mit SciPy

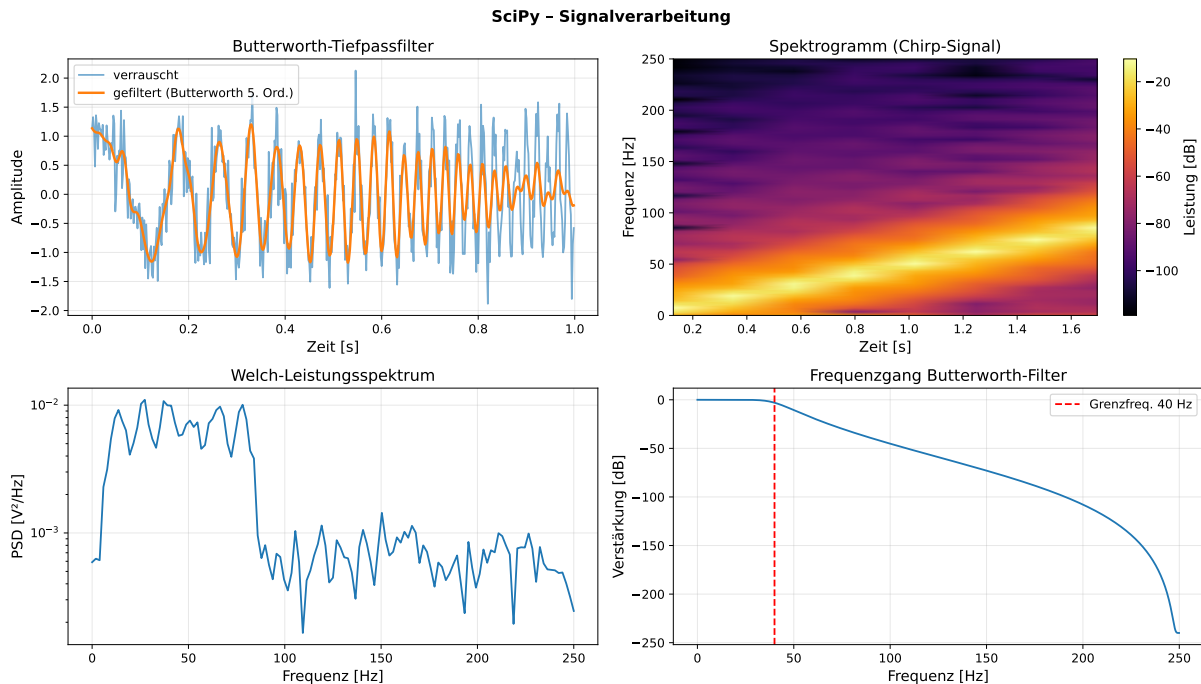


Abbildung 6: SciPy Signalverarbeitung: Butterworth-Tiefpass (oben links), Spektrogramm des Chirp-Signals (oben rechts), Welch-PSD (unten links), Filterfrequenzgang (unten rechts).

3.6 Wahrscheinlichkeitsstatistik

`scipy.stats` enthält über 100 kontinuierliche und diskrete Wahrscheinlichkeitsverteilungen sowie eine Vielzahl statistischer Tests:

```

1 from scipy import stats
2 import numpy as np
3
4 rng = np.random.default_rng(99)

```



```

5 sample1 = rng.normal(5, 1.5, 200)
6 sample2 = rng.normal(6, 2.0, 200)
7
8 # Lageparameter und Streuung
9 print(f"Mittelwert: {np.mean(sample1):.3f}")
10 print(f"Median:    {np.median(sample1):.3f}")
11 print(f"IQR:      {stats.iqr(sample1):.3f}")
12 print(f"Schiefe:   {stats.skew(sample1):.3f}")
13 print(f"Kurtosis: {stats.kurtosis(sample1):.3f}")
14
15 # t-Test (zweiseitig, unabhaengige Stichproben)
16 t_stat, p_value = stats.ttest_ind(sample1, sample2)
17 print(f"t-Test: t={t_stat:.3f}, p={p_value:.4f}")
18
19 # Wilcoxon-Mann-Whitney-U-Test (nicht-parametrisch)
20 u_stat, p_mw = stats.mannwhitneyu(sample1, sample2, alternative='two-sided'
21 )
22
23 # Kolmogorov-Smirnov-Test auf Normalverteilung
24 ks_stat, p_ks = stats.kstest(sample1, 'norm',
25                               args=(np.mean(sample1), np.std(sample1)))
26
27 # Shapiro-Wilk-Test
28 sw_stat, p_sw = stats.shapiro(sample1[:50]) # max 50 Datenpunkte empfohlen
29
30 # Pearson-Korrelation
31 r_pearson, p_pearson = stats.pearsonr(sample1, sample2)
32
33 # Spearman-Rangkorrelation
34 r_spearman, p_spearman = stats.spearmanr(sample1, sample2)
35
36 # Kernel-Dichteschaetzung (KDE)
37 kde = stats.gaussian_kde(sample1, bw_method='scott')
38 x_eval = np.linspace(0, 12, 400)
39 pdf_est = kde(x_eval)
40
41 # Anpassungstest: beste Verteilung finden
42 list_dists = [stats.norm, stats.lognorm, stats.gamma, stats.beta]
43 for dist in list_dists:
44     fit_params = dist.fit(sample1)
45     ks, p = stats.kstest(sample1, dist.name, args=fit_params)
46     print(f"{dist.name}: KS={ks:.4f}, p={p:.4f}")

```

Listing 12: Statistische Analysen mit SciPy

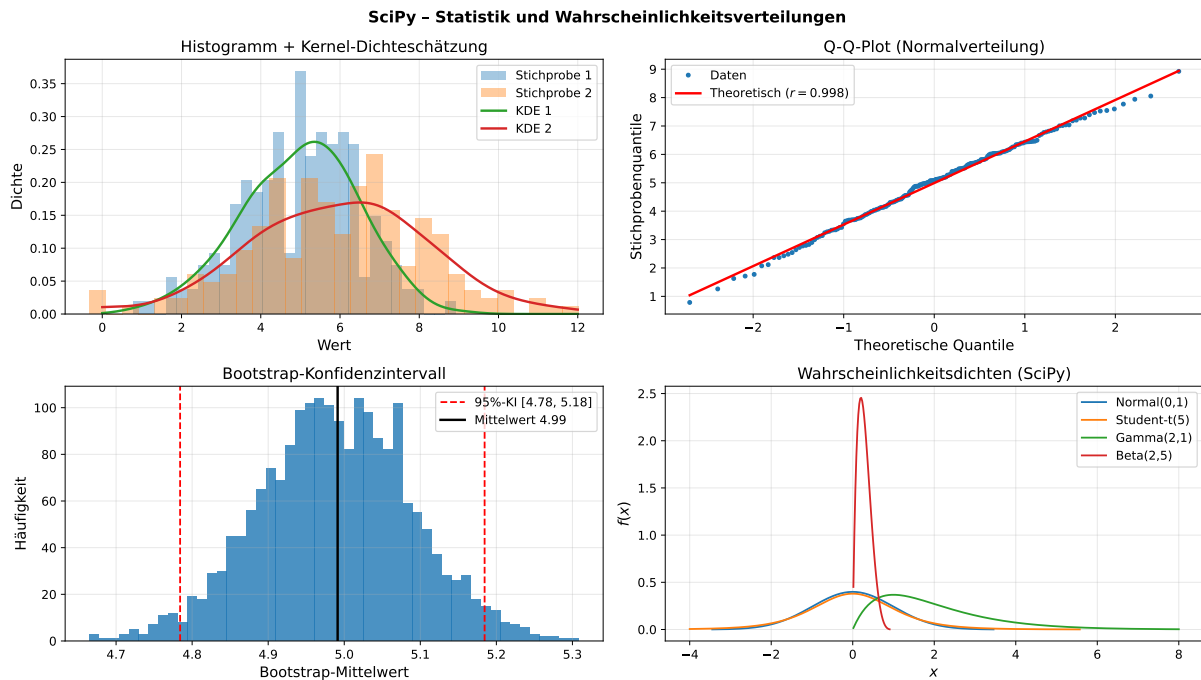


Abbildung 7: SciPy Statistik: KDE und Histogramm (oben links), Q-Q-Plot (oben rechts), Bootstrap-Konfidenzintervall (unten links), Wahrscheinlichkeitsdichten (unten rechts).

3.7 Interpolation

`scipy.interpolate` bietet eine Vielzahl von Interpolationsverfahren für ein- und mehrdimensionale Daten:

```

1 from scipy import interpolate
2 import numpy as np
3
4 # --- 1D Spline-Interpolation ---
5 x_pts = np.array([0, 1, 2, 3, 4, 5, 6])
6 y_pts = np.array([0, 0.8, 0.9, 0.1, -0.8, -1, -0.4])
7
8 f_lin = interpolate.interp1d(x_pts, y_pts, kind='linear')
9 f_cubic = interpolate.interp1d(x_pts, y_pts, kind='cubic')
10 f_quad = interpolate.interp1d(x_pts, y_pts, kind='quadratic')
11
12 # PCHIP: erhält Monotonie
13 f_pchip = interpolate.PchipInterpolator(x_pts, y_pts)
14
15 # Baryzentrischer Lagrange-Interpolation
16 f_bary = interpolate.BarycentricInterpolator(x_pts, y_pts)
17
18 # UnivariateSpline mit Glaettungsparameter
19 f_us = interpolate.UnivariateSpline(x_pts, y_pts, s=0.5)
20
21 # --- 2D RBF (Radial Basis Function) ---
22 from scipy.interpolate import RBFInterpolator

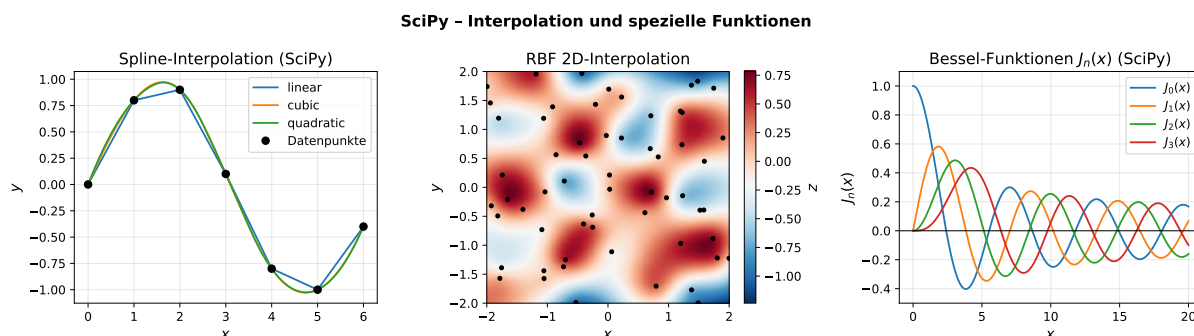
```

```

23 rng = np.random.default_rng(5)
24 xy_pts = rng.uniform(-2, 2, (60, 2))
25 z_pts = np.sin(np.pi*xy_pts[:,0]) * np.cos(np.pi*xy_pts[:,1])
26 rbf = RBFInterpolator(xy_pts, z_pts, kernel='thin_plate_spline')
27
28 # Auswertung auf Gitter
29 xg = np.linspace(-2, 2, 80)
30 yg = np.linspace(-2, 2, 80)
31 XG, YG = np.meshgrid(xg, yg)
32 ZG = rbf(np.column_stack([XG.ravel(), YG.ravel()])).reshape(80, 80)

```

Listing 13: Interpolation mit SciPy

Abbildung 8: SciPy Interpolation: 1D-Splines (links), 2D-RBF-Interpolation (Mitte), Bessel-Funktionen $J_n(x)$ (rechts).

3.8 Spezielle Funktionen der mathematischen Physik

`scipy.special` enthält Implementierungen der wichtigsten Funktionen der mathematischen Physik, Ingenieurwissenschaften und Statistik [31]:

```

1 from scipy import special
2 import numpy as np
3
4 x = np.linspace(0, 20, 500)
5
6 # Bessel-Funktionen (Zylindergeometrie, Wellenausbreitung)
7 J0 = special.jv(0, x) # Bessel 1. Art, Ordnung 0
8 J1 = special.jv(1, x)
9 Y0 = special.yv(0, x) # Bessel 2. Art
10
11 # Sphärische Besselfunktionen
12 j0_s = special.spherical_jn(0, x)
13
14 # Gamma-Funktion (Verallgemeinerung der Fakultät)
15 z = np.linspace(0.1, 5, 400)
16 Gz = special.gamma(z)
17 Lz = special.loggamma(z) # numerisch stabiler für grosse Argumente
18
19 # Legendre-Polynome (Kugelflächenfunktionen, Physik)
20 for n in range(5):

```

```

21     Pn = special.eval_legendre(n, np.linspace(-1, 1, 200))
22
23     # Elliptische Integrale (Pendel, Gravitationslinsen)
24     K = special.ellipk(0.5) # vollstaendiges elliptisches Integral K(k)
25     E = special.ellipe(0.5) # vollstaendiges elliptisches Integral E(k)
26
27     # Fehler-Funktion (Wärme-/Diffusionsgleichungen)
28     erf_vals = special.erf(np.linspace(-3, 3, 200))
29
30     # Hypergeometrische Funktionen
31     hyp2f1 = special.hyp2f1(1, 2, 3, 0.5) # Gauss'sche hypergeom. Fkt.
32
33     # Sphärische Harmonische  $Y_l^m$ 
34     theta = np.pi/4; phi = np.pi/3
35     Y11 = special.sph_harm(1, 1, phi, theta)

```

Listing 14: Spezielle Funktionen mit SciPy

3.9 Einzigartiger Wert von SciPy

SciPy eliminiert die Notwendigkeit, hochkomplexe numerische Algorithmen selbst zu implementieren. Die enthaltenen Verfahren basieren auf Jahrzehnten mathematischer Forschung (LAPACK, FITPACK, MINPACK, QUADPACK u. a.) und sind in C, Fortran oder C++ implementiert – mit Python-Schnittstelle. Ob Differentialgleichungen in der Biophysik, Signalfilterung in der Neurologie oder Optimierungsprobleme in der Luft- und Raumfahrt: SciPy stellt den direkten Zugang bereit.

4 Matplotlib – Wissenschaftliche Visualisierung

4.1 Architektur und Designphilosophie

Matplotlib wurde 2003 von John D. Hunter entwickelt, um MATLAB-ähnliche Plotting-Fähigkeiten in Python bereitzustellen [8]. Die Architektur ist in drei Schichten gegliedert:

1. **Backend-Schicht:** Zuständig für das eigentliche Rendering (Agg, PDF, SVG, Qt5Agg, WebAgg, ...).
2. **Artist-Schicht:** Alle sichtbaren Objekte (Figure, Axes, Line2D, Patch, ...).
3. **Scripting-Schicht** (pyplot): Zustandsbehaftete Komfort-API, ähnlich MATLAB.

```

1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  fig, axes = plt.subplots(2, 3, figsize=(14, 8))
5  fig.suptitle('Wissenschaftliche Abbildung', fontsize=14, fontweight='bold')
6
7  x = np.linspace(0, 4*np.pi, 500)
8
9  # Linienstil, Farbe, Label

```

```

10 axes[0, 0].plot(x, np.sin(x), lw=2, color='tab:blue', label='$\sin(x)$')
11 axes[0, 0].plot(x, np.cos(x), lw=2, color='tab:orange', linestyle='--',
12                 label='$\cos(x)$')
13 axes[0, 0].set_title('Trigonometrische Funktionen')
14 axes[0, 0].set_xlabel('$x$'); axes[0, 0].set_ylabel('Amplitude')
15 axes[0, 0].legend()
16 axes[0, 0].grid(True, alpha=0.3)
17
18 # Feinabstimmung: Tick-Formatierung
19 import matplotlib.ticker as ticker
20 axes[0, 0].xaxis.set_major_locator(ticker.MultipleLocator(np.pi))
21 axes[0, 0].xaxis.set_major_formatter(
22     ticker.FuncFormatter(lambda x, _: f'${x/np.pi:.0f}\\pi$')
23 )
24
25 plt.tight_layout()
26 plt.savefig('output.pdf', dpi=300, bbox_inches='tight')

```

Listing 15: Objektbasierte Matplotlib-API (empfohlen)

4.2 Grundlegende und erweiterte Diagrammtypen

Abbildung 9 zeigt eine Auswahl der wichtigsten Diagrammtypen:

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 rng = np.random.default_rng(42)
5
6 # Polarplot
7 fig, ax = plt.subplots(subplot_kw={'projection': 'polar'})
8 theta = np.linspace(0, 2*np.pi, 1000)
9 ax.plot(theta, np.abs(np.cos(4*theta)), label='Rosenplot')
10
11 # Histogramm mit Kerndichteschätzung
12 fig, ax = plt.subplots()
13 data = rng.normal(0, 1, 1000)
14 ax.hist(data, bins=40, density=True, alpha=0.6,
15         edgecolor='white', label='Histogramm')
16
17 # Boxplot mit Notch (Konfidenzintervall)
18 data_groups = [rng.normal(i, 1, 100) for i in range(5)]
19 fig, ax = plt.subplots()
20 bp = ax.boxplot(data_groups, notch=True, patch_artist=True,
21                 medianprops=dict(color='red', linewidth=2))
22 for patch in bp['boxes']:
23     patch.set_facecolor('lightblue')
24
25 # Violin Plot
26 fig, ax = plt.subplots()
27 vp = ax.violinplot(data_groups, showmedians=True, showextrema=True)

```

```

28
29 # Stem Plot (diskrete Signale)
30 x_s = np.linspace(-np.pi, np.pi, 25)
31 fig, ax = plt.subplots()
32 ax.stem(x_s, np.sinc(x_s))
33
34 # Heatmap (Korrelationsmatrix)
35 corr = np.corrcoef(rng.normal(size=(5, 200)))
36 fig, ax = plt.subplots()
37 im = ax.imshow(corr, cmap='coolwarm', vmin=-1, vmax=1)
38 plt.colorbar(im, ax=ax)

```

Listing 16: Vielfalt der Matplotlib-Diagrammtypen

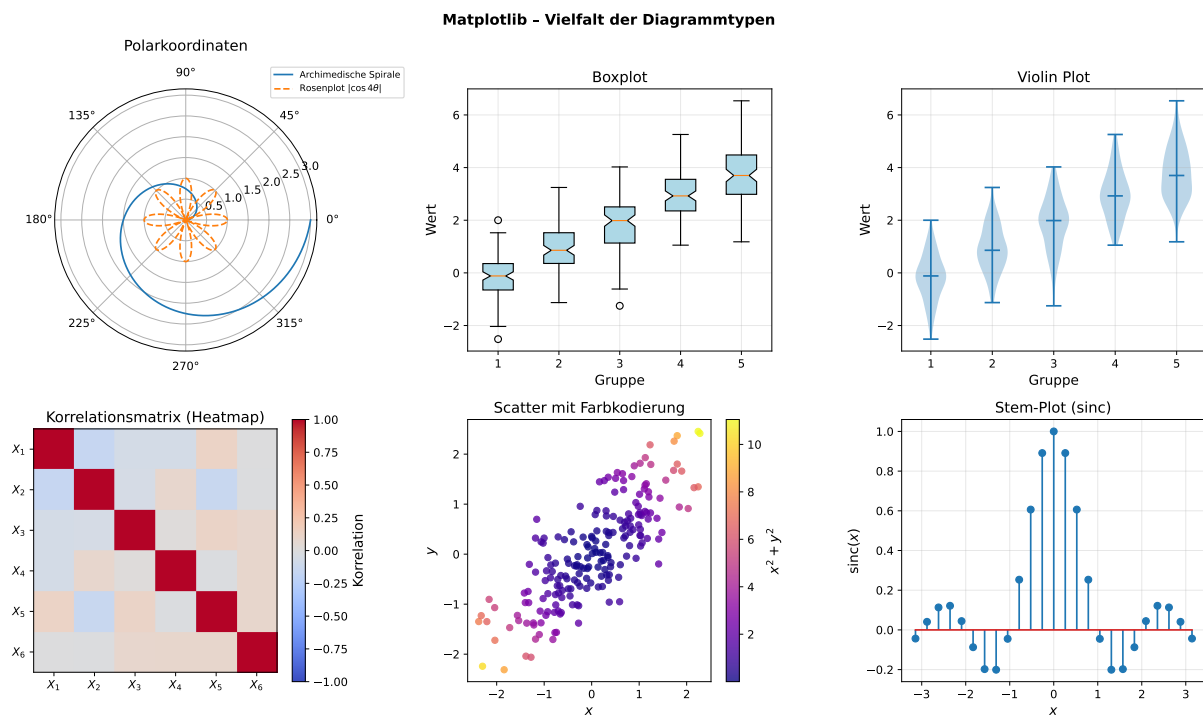


Abbildung 9: Matplotlib Diagrammvielfalt: Polarplot, Boxplot, Violin Plot, Korrelationsheatmap, Scatter mit Farbkodierung, Stem-Plot.

4.3 3D-Visualisierung

Das Toolkit `mpl_toolkits.mplot3d` ermöglicht reichhaltige dreidimensionale Darstellungen:

```

1 from mpl_toolkits.mplot3d import Axes3D
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 fig = plt.figure(figsize=(15, 5))
6
7 # 3D-Oberfläche
8 ax1 = fig.add_subplot(131, projection='3d')

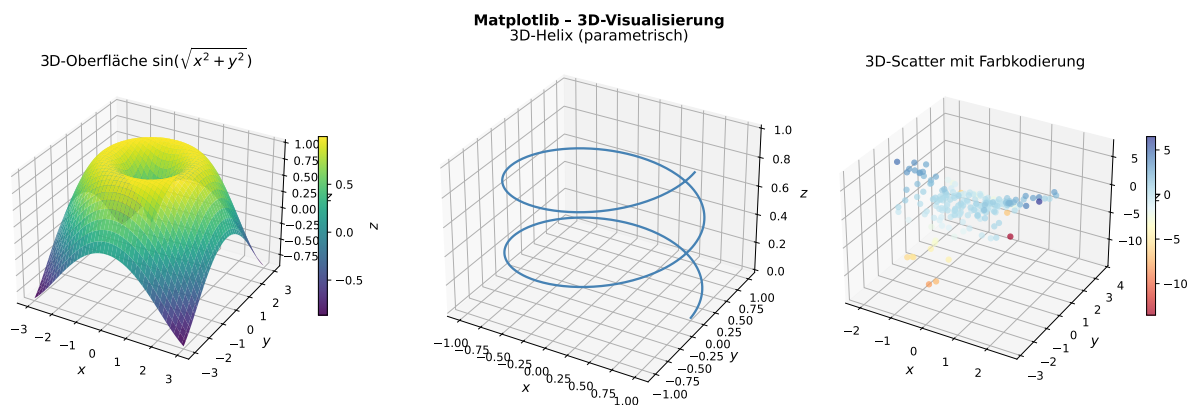
```

```

9  x = y = np.linspace(-3, 3, 80)
10 X, Y = np.meshgrid(x, y)
11 Z = np.sin(np.sqrt(X**2 + Y**2))
12 surf = ax1.plot_surface(X, Y, Z, cmap='viridis',
13                        edgecolor='none', antialiased=True)
14 fig.colorbar(surf, ax=ax1, shrink=0.5)
15
16 # Drahtgitter-Plot
17 ax1b = fig.add_subplot(131, projection='3d')
18 ax1b.plot_wireframe(X, Y, Z, rstride=5, cstride=5, linewidth=0.3)
19
20 # 3D-Kontur
21 ax1c = fig.add_subplot(131, projection='3d')
22 ax1c.contourf(X, Y, Z, zdir='z', levels=20, cmap='plasma')
23
24 # Parametrische Kurve (Helix)
25 ax2 = fig.add_subplot(132, projection='3d')
26 t = np.linspace(0, 4*np.pi, 500)
27 ax2.plot(np.cos(t), np.sin(t), t/(4*np.pi), lw=2)
28
29 # 3D-Scatter
30 ax3 = fig.add_subplot(133, projection='3d')
31 rng = np.random.default_rng(11)
32 xs = rng.normal(0, 1, 200)
33 ys = rng.normal(0, 1, 200)
34 zs = xs**2 - ys**2
35 ax3.scatter(xs, ys, zs, c=zs, cmap='RdYlBu', s=20, alpha=0.7)

```

Listing 17: 3D-Visualisierung mit Matplotlib

Abbildung 10: Matplotlib 3D: Oberfläche $\sin(\sqrt{x^2 + y^2})$ (links), 3D-Helix als parametrische Kurve (Mitte), 3D-Scatter- Diagramm mit Farbkodierung (rechts).

4.4 Kontur-, Vektorfeld- und Stromlinienplots

```

1  import matplotlib.pyplot as plt
2  import numpy as np
3

```

```

4 x = np.linspace(-3, 3, 200)
5 y = np.linspace(-3, 3, 200)
6 X, Y = np.meshgrid(x, y)
7 Z = X * np.exp(-(X**2 + Y**2))
8
9 fig, axes = plt.subplots(1, 3, figsize=(14, 4))
10
11 # Gefüllter Konturplot
12 cf = axes[0].contourf(X, Y, Z, levels=20, cmap='RdBu_r')
13 axes[0].contour(X, Y, Z, levels=20, colors='k', linewidths=0.5)
14 plt.colorbar(cf, ax=axes[0])
15
16 # Quiver-Plot (Vektorfeld)
17 xq = np.linspace(-2, 2, 20); yq = np.linspace(-2, 2, 20)
18 XQ, YQ = np.meshgrid(xq, yq)
19 speed = np.sqrt((-YQ)**2 + XQ**2)
20 axes[1].quiver(XQ, YQ, -YQ, XQ, speed, cmap='autumn')
21
22 # Streamplot (Flusslinien)
23 xs = np.linspace(-3, 3, 100); ys = np.linspace(-3, 3, 100)
24 XS, YS = np.meshgrid(xs, ys)
25 speed_s = np.sqrt((-YS)**2 + XS**2)
26 axes[2].streamplot(xs, ys, -YS, XS, color=speed_s,
27                   cmap='Blues', linewidth=1.5, density=1.5)

```

Listing 18: Kontur-, Quiver- und Streamplot

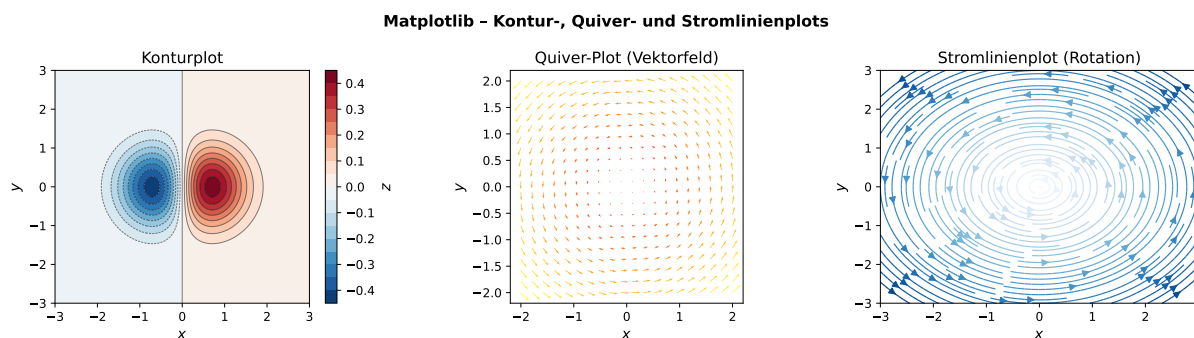


Abbildung 11: Matplotlib: Konturplot (links), Quiver-Vektorfeld (Mitte), Stromlinienplot einer Rotationsströmung (rechts).

4.5 Annotationen und Unsicherheitsdarstellung

Für wissenschaftliche Publikationen unverzichtbar ist die Darstellung von Messunsicherheiten und die gezielte Beschriftung relevanter Merkmale:

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
5
6 # Unsicherheitsband

```

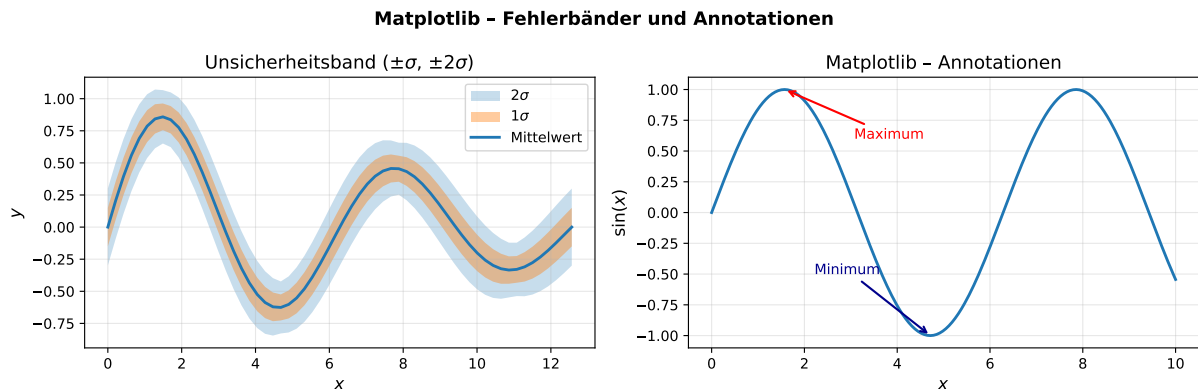


```

7 x = np.linspace(0, 4*np.pi, 100)
8 y_mean = np.sin(x) * np.exp(-0.1 * x)
9 y_std = 0.1 + 0.05 * np.abs(np.cos(x))
10
11 axes[0].fill_between(x, y_mean - 2*y_std, y_mean + 2*y_std,
12                     alpha=0.25, label='$2\sigma$')
13 axes[0].fill_between(x, y_mean - y_std, y_mean + y_std,
14                     alpha=0.4, label='$1\sigma$')
15 axes[0].plot(x, y_mean, lw=2)
16
17 # Annotationen mit Pfeilen
18 y = np.sin(np.linspace(0, 10, 300))
19 max_idx = np.argmax(y)
20 axes[1].plot(np.linspace(0, 10, 300), y, lw=2)
21 axes[1].annotate('Maximum',
22                 xy=(np.linspace(0, 10, 300)[max_idx], y[max_idx]),
23                 xytext=(np.linspace(0, 10, 300)[max_idx]+1.5, 0.6),
24                 arrowprops=dict(arrowstyle='->', color='red', lw=1.5),
25                 fontsize=11, color='red')
26 )
27
28 # Mathematische Formeln als Text
29 axes[1].text(0.5, -0.8, r'$y = \sin(x)$',
30             fontsize=13, transform=axes[1].transAxes,
31             ha='center')

```

Listing 19: Annotationen und Fehlerbänder in Matplotlib

Abbildung 12: Matplotlib: $\pm\sigma$ - und $\pm2\sigma$ -Unsicherheitsbänder (links), Annotationen mit Pfeilen (rechts).

4.6 Farbpaletten (Colormaps)

Die Wahl der Farbpalette ist entscheidend für die korrekte Wahrnehmung wissenschaftlicher Daten. Matplotlib bietet über 80 Paletten, darunter perzeptuell gleichförmige Karten wie `viridis`, `plasma`, `inferno` und `cividis` [8]:

```

1 import matplotlib.pyplot as plt
2 import matplotlib.cm as cm

```

```

3 import numpy as np
4
5 # Alle verfügbaren Colormaps anzeigen
6 print(plt.colormaps())
7
8 x = y = np.linspace(-np.pi, np.pi, 100)
9 X, Y = np.meshgrid(x, y)
10 Z = np.sin(X) * np.cos(Y)
11
12 cmaps = ['viridis', 'plasma', 'inferno', 'magma',
13          'cividis', 'RdBu_r', 'coolwarm', 'nipy_spectral']
14
15 fig, axes = plt.subplots(2, 4, figsize=(16, 6))
16 for ax, cmap in zip(axes.ravel(), cmaps):
17     ax.imshow(Z, cmap=cmap, origin='lower')
18     ax.set_title(cmap)
19     ax.axis('off')
20
21 # Benutzerdefinierte Colormap
22 from matplotlib.colors import LinearSegmentedColormap
23 custom_cmap = LinearSegmentedColormap.from_list(
24     'mein_cm', ['white', 'steelblue', 'darkblue'])
25 )

```

Listing 20: Colormaps und barrierefreie Farbgebung

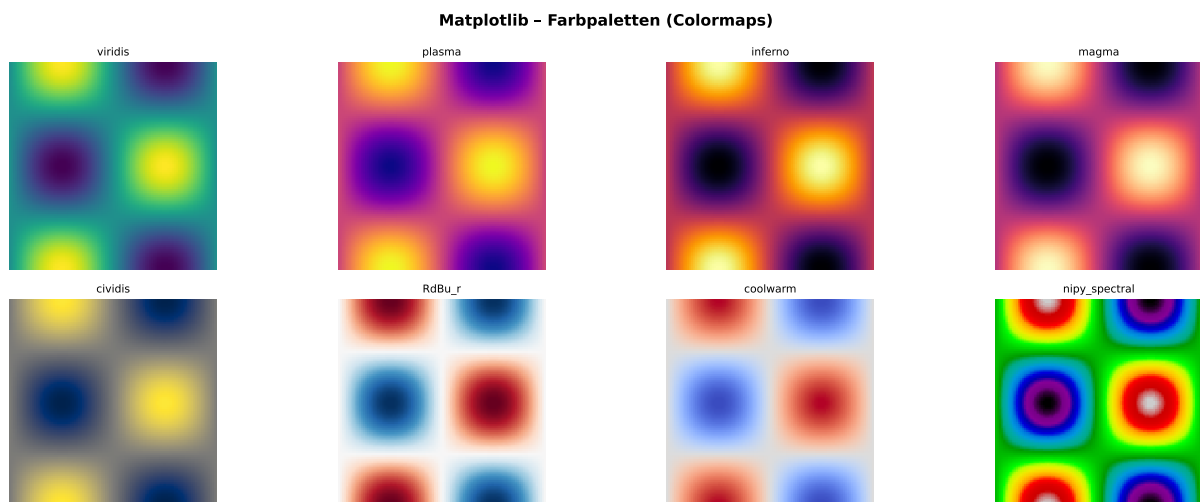


Abbildung 13: Matplotlib Farbpaletten: Viridis, Plasma, Inferno, Magma, Cividis, RdBu_r, Coolwarm und Nipy_Spectral.

4.7 Einzigartiger Wert von Matplotlib

Matplotlib ist das einzige Python-Visualisierungswerkzeug, das gleichzeitig *vollständige* Kontrolle über jeden Aspekt einer Grafik bietet und dabei in allen wissenschaftlichen Backends (PDF, SVG, EPS für Journale, PNG/TIFF für Poster) erstklassige Ausgaben erzeugt. Keine andere Bibliothek erreicht vergleichbare typografische Qualität bei der

Ausgabe von LaTeX-gerendertem Text und mathematischen Formeln (`usetex=True` oder `mathtext`).

5 Zusammenspiel der Bibliotheken: Anwendungsbeispiele

5.1 Nichtlineare Dynamik: Das Lorenz-System

Das Lorenz-System [27] ist das Paradigma des deterministischen Chaos:

$$\dot{x} = \sigma(y - x), \quad (3)$$

$$\dot{y} = x(\rho - z) - y, \quad (4)$$

$$\dot{z} = xy - \beta z, \quad (5)$$

mit den klassischen Parametern $\sigma = 10$, $\rho = 28$, $\beta = 8/3$.

```

1 import numpy as np
2 from scipy import integrate
3 import matplotlib.pyplot as plt
4
5 def lorenz(t, state, sigma=10, rho=28, beta=8/3):
6     x, y, z = state
7     return [sigma*(y-x), x*(rho-z)-y, x*y-beta*z]
8
9 # Numerische Lösung (SciPy solve_ivp)
10 sol = integrate.solve_ivp(
11     lorenz, (0, 50), [1, 1, 1],
12     t_eval=np.linspace(0, 50, 20000),
13     method='RK45', max_step=0.01
14 )
15
16 # Visualisierung (Matplotlib)
17 fig = plt.figure(figsize=(12, 5))
18 ax1 = fig.add_subplot(121, projection='3d')
19 ax1.plot(sol.y[0], sol.y[1], sol.y[2], lw=0.3, alpha=0.7)
20 ax1.set_title('Lorenz-Attraktor (3D)')
21
22 # Leistungsspektrum (NumPy FFT)
23 ax2 = fig.add_subplot(122)
24 freqs = np.fft.rfftfreq(len(sol.t), d=50/20000)
25 power = np.abs(np.fft.rfft(sol.y[0]))**2
26 ax2.semilogy(freqs[1:1000], power[1:1000])
27 ax2.set_title('Leistungsspektrum (Lorenz $x$)')
```

Listing 21: Lorenz-Attraktor mit NumPy, SciPy und Matplotlib

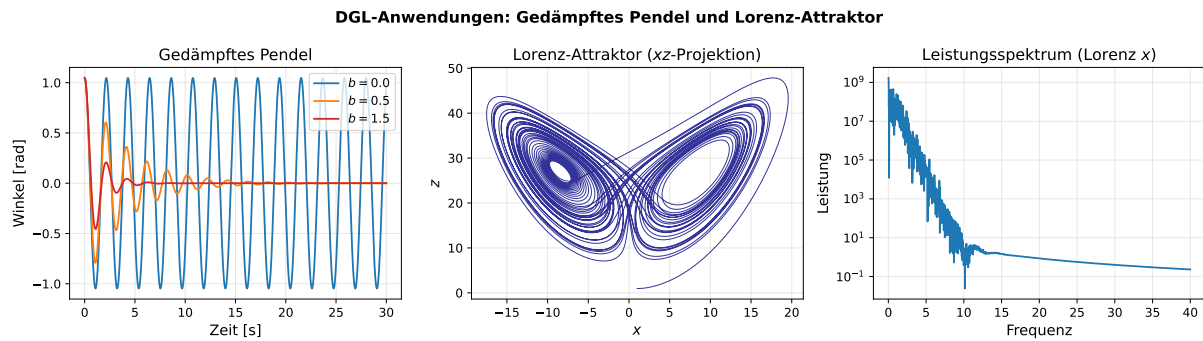


Abbildung 14: DGL-Anwendungen: Gedämpftes Pendel (links), Lorenz-Attraktor (Mitte), Leistungsspektrum der x -Komponente (rechts).

5.2 Wärmeleitungsgleichung: Spektralmethode

Die eindimensionale Wärmeleitungsgleichung

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2} \quad (6)$$

kann mit NumPys FFT exakt und effizient (spektrale Methode) gelöst werden:

```

1 import numpy as np
2
3 N = 256; L = 2*np.pi; alpha = 0.01
4 x = np.linspace(0, L, N, endpoint=False)
5 u = np.exp(-4*(x - np.pi)**2) # Gauss-Anfangsbedingung
6
7 k = np.fft.rfftfreq(N) * N * (2*np.pi/L) # Wellenzahlen
8 dt = 0.01; n_steps = 500
9 u_hat = np.fft.rfft(u)
10
11 snapshots = [u.copy()]
12 for i in range(n_steps):
13     u_hat *= np.exp(-alpha * k**2 * dt) # exakte spektrale Propagation
14     if (i+1) % 100 == 0:
15         snapshots.append(np.fft.irfft(u_hat))
16
17 # Bildverarbeitung: Gauss-Glättung als Wärmediffusion
18 from scipy.ndimage import gaussian_filter
19 image = np.zeros((100, 100))
20 image[30:70, 30:70] = 1.0
21 blurred = gaussian_filter(image, sigma=5)

```

Listing 22: PDE-Loesung via FFT (NumPy)

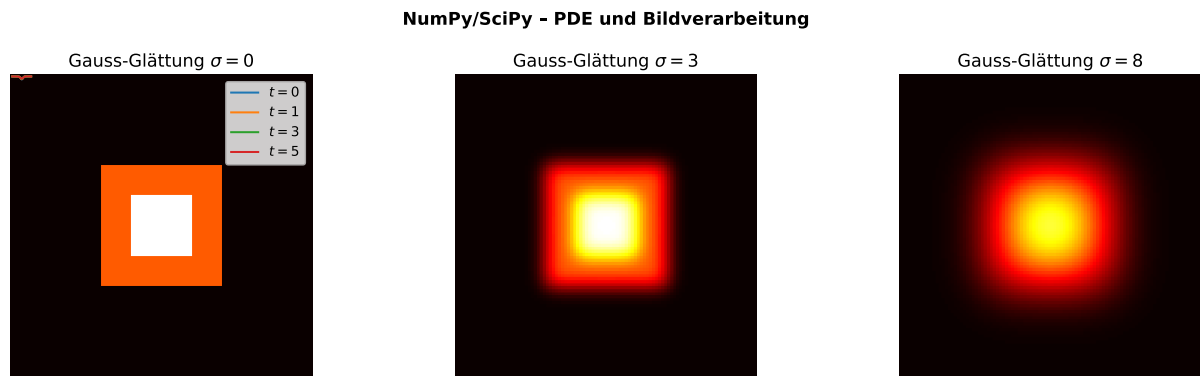


Abbildung 15: PDE und Bildverarbeitung: Gauss-Glättung mit $\sigma = 0, 3, 8$ als Analogie zur Wärmeleitung.

5.3 Zusammenfassende Bewertung

Aufgabe	NumPy	SciPy	Matplotlib
Array-Operationen	✓✓✓	✓	–
FFT/Spektralanalyse	✓✓	✓✓✓	–
Lineare Algebra	✓✓	✓✓✓	–
Optimierung	–	✓✓✓	–
DGL-Lösung	–	✓✓✓	–
Statistik	✓	✓✓✓	–
Interpolation	✓	✓✓✓	–
2D-Plots	–	–	✓✓✓
3D-Plots	–	–	✓✓
Publikationsqualität	–	–	✓✓✓

Tabelle 2: Stärken (✓✓✓ = hauptverantwortlich, ✓ = unterstützend, – = nicht primär) der drei Bibliotheken in den wichtigsten wissenschaftlichen Aufgabenbereichen.

6 Leistungsmessung und Skalierbarkeit

Ein wesentlicher Vorteil dieser Bibliotheken liegt in ihrer Effizienz. Tabelle 3 zeigt exemplarische Geschwindigkeitsvergleiche zwischen reinem Python und NumPy/SciPy:

Operation	Reines Python	NumPy/SciPy	Speedup
Summe (10^7 Elemente)	ca. 1 000 ms	ca. 8 ms	$\approx 125\times$
Matmul (1000×1000)	ca. 300 s	ca. 0.06 s	$\approx 5\,000\times$
FFT (2^{20} Punkte)	N/A (manuell)	ca. 120 ms	$\gg$
Sortierung (10^6)	ca. 800 ms	ca. 60 ms	$\approx 13\times$
ODE-Lösung (RK45)	ca. 60 s	ca. 0.1 s	$\approx 600\times$

Tabelle 3: Exemplarische Laufzeitvergleiche auf einem modernen Laptop (Intel i7, 16 GB RAM). Die Werte sind Richtwerte und können je nach Hardware variieren [4].

7 Schlussfolgerung

Die vorliegende Arbeit hat gezeigt, dass **NumPy**, **SciPy** und **Matplotlib** zusammen eine vollständige, hochleistungsfähige und ausdrucksstarke Infrastruktur für das wissenschaftliche Rechnen mit Python bilden. Ihre Bedeutung lässt sich in drei Dimensionen fassen:

1. **Ausdrucksstärke:** Die Bibliotheken ermöglichen die Formulierung komplexer mathematischer Konzepte (Eigenwertzerlegungen, DGL-Systeme, Faltungsfilter, spezielle Funktionen) in wenigen, lesbaren Python-Zeilen.
2. **Effizienz:** Durch C/Fortran-Backends und Algorithmen aus jahrzehntelanger numerischer Forschung sind die Implementierungen oft mit spezialisierter Forschungssoftware vergleichbar oder überlegen.
3. **Ökosystem:** Als Bausteine von Pandas, scikit-learn, TensorFlow, Astropy, BioPython u. v. a. sind diese drei Bibliotheken die gemeinsame Sprache der wissenschaftlichen Python-Gemeinschaft.

Die Anwendungsbeispiele – vom Lorenz-Attraktor über die spektrale PDE-Lösung bis hin zu statistischen Tests und Signalfiltern – belegen eindrücklich, dass das wissenschaftliche Arbeiten ohne das Trio NumPy/SciPy/Matplotlib in der heutigen Form kaum vorstellbar wäre. Zukünftige Entwicklungen wie CuPy (GPU-NumPy), JAX (differenzierbares NumPy) und SciPy.fft (FFTW-Backend) werden auf demselben konzeptionellen Fundament aufbauen und zeigen, dass die hier dargelegten Prinzipien zeitlos und zukunftssicher sind [41].

7.1 Ausblick: Entwicklungstendenzen und Zukunftsperspektiven

Die vorstehende Analyse hat gezeigt, dass **NumPy**, **SciPy** und **Matplotlib** in ihrer heutigen Form bereits einen außerordentlichen Reifegrad besitzen. Gleichwohl ist das wissenschaftliche Python-Ökosystem kein statisches Gebilde, sondern ein dynamisches, sich kontinuierlich weiterentwickelndes System, das auf Herausforderungen der Praxis reagiert. Im Folgenden werden die bedeutendsten Entwicklungstendenzen und Zukunftsperspektiven ausführlich dargelegt, die das wissenschaftliche Rechnen mit Python in den kommenden Jahren grundlegend prägen werden.

7.1.1 GPU-Beschleunigung und heterogenes Rechnen

Die rasante Verbreitung von Grafikprozessoren (GPUs) in Forschung und Industrie hat eine neue Klasse von NumPy-kompatiblen Bibliotheken hervorgebracht, die nahezu identische Schnittstellen bieten, die Berechnungen aber auf GPU-Hardware ausführen.

CuPy [42] stellt eine direkte Drop-in-Ersetzung für NumPy auf NVIDIA-CUDA-Hardware bereit. Für Matrizen der Größe $10\,000 \times 10\,000$ erreicht CuPy bei Matrixmultiplikationen Speedups von $100\times$ gegenüber NumPy auf der CPU. Die Migration eines bestehenden NumPy-Codes erfordert in vielen Fällen lediglich den Austausch des Import-Statements:

```
1 # Vorher: import numpy as np
2 import cupy as np # identische API, GPU-Ausführung
```

```

3
4 A = np.random.normal(size=(5000, 5000))
5 B = np.linalg.svd(A) # laeuft auf GPU

```

Listing 23: CuPy als Drop-in-Ersatz fuer NumPy

ROCm/HIP (AMD) und **Metal** (Apple Silicon) erweitern den Horizont über NVIDIA-Hardware hinaus. Bibliotheken wie **PyOpenCL** und **ArrayFire** erlauben hardwareagnostisches GPU-Computing. Mittelfristig ist zu erwarten, dass SciPy und NumPy selbst optionale GPU-Backends über das standardisierte Array-API-Protokoll (siehe Abschnitt 7.1.6) unterstützen werden.

Neben der GPU-Beschleunigung gewinnt auch das sogenannte *heterogene Rechnen* an Bedeutung: Rechenaufgaben werden dabei dynamisch auf die optimale Hardwareeinheit verteilt – CPU-Kerne für seriell-sequentielle Teilprobleme, GPUs für massiv-parallele Matrix- und Tensoroperationen, und spezialisierte Beschleuniger wie FPGAs oder TPUs für besonders häufig wiederkehrende Routinen. Die Fähigkeit von NumPy, als gemeinsame konzeptuelle Sprache für all diese Plattformen zu dienen, macht das Array zu einem universellen Abstraktionsinstrument.

7.1.2 JAX: Differenzierbares Programmieren und JIT-Kompilierung

JAX [41] von Google DeepMind ist die bislang weitreichendste Erweiterung des NumPy-Paradigmas. Es kombiniert vier transformative Fähigkeiten in einer einzigen Bibliothek:

1. **Automatische Differentiation (`jax.grad`):** Jede in NumPy-Stil geschriebene Funktion kann automatisch abgeleitet werden – sowohl Vorwärts- als auch Rückwärtsmodus. Dies ermöglicht die Implementierung von Gradientenabstiegsverfahren, adjungierten Methoden für PDE-Optimierung und Sensitivitätsanalysen, ohne dass analytische Ableitungen manuell hergeleitet werden müssen.
2. **JIT-Kompilierung (`jax.jit`):** Funktionen werden bei erstem Aufruf durch die XLA-Compiler-Pipeline übersetzt und erreichen dadurch Ausführungsgeschwindigkeiten nahe am theoretischen Hardware-Limit.
3. **Vektorisierung (`jax.vmap`):** Eine für ein einzelnes Element geschriebene Funktion wird automatisch auf ganze Batches von Eingaben vektorisiert, ohne explizite Schleifen.
4. **Parallelisierung (`jax.pmap`):** Code wird automatisch auf mehrere GPU- oder TPU-Einheiten verteilt.

```

1 import jax
2 import jax.numpy as jnp
3
4 def potential(q):
5     """Harmonisches Potential:  $V(q) = 0.5 * q^2$ """
6     return 0.5 * jnp.dot(q, q)
7
8 # Gradient dV/dq analytisch korrekt
9 grad_V = jax.grad(potential)
10

```

```

11 # Hessematrix  $d^2V/dq^2$ 
12 hess_V = jax.jacfwd(jax.grad(potential))
13
14 # JIT-Kompilierung
15 @jax.jit
16 def simulation_step(q, p, dt=0.01):
17     force = -grad_V(q)
18     p_new = p + dt * force
19     q_new = q + dt * p_new
20     return q_new, p_new
21
22 # Vektorisierung ueber 1000 Trajektorien gleichzeitig
23 batched_step = jax.vmap(simulation_step)

```

Listing 24: JAX: Automatische Differentiation und JIT

Die Konsequenz dieser Fähigkeiten ist fundamental: Physikalische Modelle, die zuvor nur als *Forward-Simulationen* ausgeführt werden konnten, werden durch JAX zu differenzierbaren Programmen, deren Parameter durch gradientenbasierte Optimierungsverfahren kalibriert werden können. Dies öffnet die Tür zu einem neuen Paradigma des *Scientific Machine Learning*, das numerische Simulation und maschinelles Lernen organisch verbindet.

7.1.3 Physik-informierte neuronale Netze (PINNs)

Eine der bedeutsamsten Entwicklungen an der Schnittstelle von maschinellem Lernen und numerischer Mathematik ist das Konzept der *Physics-Informed Neural Networks* (PINNs). Dabei werden neuronale Netze nicht nur durch Daten, sondern auch durch die Residuen bekannter partieller Differentialgleichungen (PDEs) trainiert. Das Netz approximiert die Lösung der PDE direkt als Funktion der Raumzeit-Koordinaten.

NumPy und SciPy spielen bei PINNs eine doppelte Rolle: Erstens als Werkzeuge zur Datengenerierung, Gittererstellung und Referenzlösung; zweitens als Backend für Bibliotheken wie DeepXDE oder NeuralPDE.jl, die auf NumPy-kompatiblen Array-Operationen aufbauen.

Die Navier-Stokes-Gleichungen, Schrödinger-Gleichungen, Diffusionsgleichungen und Maxwell-Gleichungen werden derzeit aktiv als PINN-Benchmarks untersucht. Zukünftig ist zu erwarten, dass SciPy-Löser als Kollokationspunktgeneratoren und Referenzlöser für den Trainingsprozess von PINNs dienen werden, während Matplotlib zur Visualisierung der gelernten Lösungsfelder unverzichtbar bleibt.

7.1.4 Numba und Cython: Just-in-Time-Kompilierung für Python

Für Algorithmen, die sich nicht vollständig vektorisieren lassen und explizite Schleifen erfordern, bieten **Numba** und **Cython** komplementäre Ansätze zur Beschleunigung:

```

1 from numba import njit, prange
2 import numpy as np
3
4 @njit(parallel=True)
5 def stencil_diffusion(u, alpha, dt, dx):
6     """Finite-Differenzen-Diskretisierung der Waermeleitung."""
7     n = len(u)

```



```

8   u_new = np.empty_like(u)
9   for i in prange(1, n - 1): # parallel ueber alle inneren Punkte
10      u_new[i] = u[i] + alpha * dt / dx**2 * (u[i+1] - 2*u[i] + u[i-1])
11   u_new[0] = u[0]           # Randbedingung links
12   u_new[-1] = u[-1]        # Randbedingung rechts
13   return u_new

```

Listing 25: Numba: JIT-Kompilierung von Python-Schleifen

Numba kompiliert die mit `@jit` dekorierten Funktionen bei erstem Aufruf durch LLVM zu nativem Maschinencode. Schleifen, die in reinem Python um den Faktor 1 000 langsamer wären als die entsprechende C-Implementierung, erreichen nach Numba-Kompilierung nahezu identische Laufzeiten. Dies ist besonders relevant für iterative PDE-Löser, Graphalgorithmen und Monte-Carlo-Simulationen, bei denen Broadcasting kein vollständiges Vektorisierungsäquivalent ermöglicht.

Mittelfristig werden Numba und NumPy noch stärker integriert werden: Das **NumPy Enhancement Proposal 47 (NEP 47)** schlägt eine standardisierte `__array_namespace__`-Schnittstelle vor, die Numba-JIT-kompilierte Funktionen transparent über beliebige Array-Backends hinweg ausführbar macht.

7.1.5 Paralleles und verteiltes Rechnen: Dask und Ray

Sobald Datensätze den verfügbaren Arbeitsspeicher überschreiten oder Berechnungen auf Cluster-Infrastrukturen verteilt werden sollen, greifen Wissenschaftler zunehmend auf **Dask** zurück. Dask stellt verteilte Äquivalente zu NumPy-Arrays und Pandas-DataFrames bereit, wobei die Berechnungen als azyklische Abhängigkeitsgraphen (DAGs) modelliert und lazy ausgewertet werden:

```

1  import dask.array as da
2  import numpy as np
3
4  # 100 GB grosses Array -- wird nie vollstaendig in den RAM geladen
5  x = da.random.normal(size=(100_000, 100_000), chunks=(1_000, 1_000))
6
7  # Berechnung wird lazy aufgebaut
8  result = (x * x).mean(axis=0)
9
10 # Erst hier: parallele Ausfuehrung auf allen CPU-Kernen
11 result_computed = result.compute()

```

Listing 26: Dask: Out-of-Core-NumPy fuer große Datensätze

Ray wiederum ermöglicht die feingranulare Verteilung beliebiger Python-Funktionen auf Cluster-Knoten und bietet mit **Ray Tune** eine skalierbare Hyperparameter-Optimierungsplattform, die intern auf SciPy-Optimierungsroutinen zurückgreift.

Xarray erweitert NumPy-Arrays um benannte Dimensionen und Koordinaten (ähnlich wie NetCDF-Dateien) und ist zur Standardbibliothek der Klima- und Ozeanografiewissenschaften geworden. In Verbindung mit Dask ermöglicht Xarray die Verarbeitung von petabytegroßen Klimasimulationsdaten.

Zu erwarten ist, dass diese Out-of-Core- und Distributed-Computing- Lösungen zunehmend direkt in SciPy und NumPy integriert werden, sodass der Übergang zwischen lokalem und verteiltem Rechnen für den Endanwender transparent und reibungslos gestaltet wird.

7.1.6 Das Array-API-Protokoll und Bibliotheks-Interoperabilität

Eines der ambitioniertesten aktuellen Vorhaben im Scientific-Python- Ökosystem ist die Standardisierung einer einheitlichen *Array-API*, die von der Python-Datenwissenschafts-Community unter dem Dach der *Python Array API Standard-Initiative* (ehemals NEP 47) vorangetrieben wird.

Das Ziel ist es, eine minimale, aber ausreichend reichhaltige Menge von Array-Operationen zu definieren, die von allen bedeutenden Array- Bibliotheken – NumPy, CuPy, JAX, PyTorch, TensorFlow, Dask, cuDF – implementiert werden. Bibliotheken wie SciPy, die auf dieser Schnittstelle aufbauen, könnten dann transparent auf allen unterstützten Backends ausgeführt werden, ohne den Quellcode zu ändern:

```

1  # Hypothetisches Beispiel fuer Array-API-transparentes SciPy
2  import scipy.linalg as la
3
4  # Dasselbe SVD laeuft auf NumPy-Array (CPU)
5  import numpy as np
6  A_cpu = np.random.normal(size=(1000, 1000))
7  U, s, Vt = la.svd(A_cpu)  # Ausfuehrung auf CPU
8
9  # ... und auf CuPy-Array (GPU) -- kein Codechange
10 import cupy as cp
11 A_gpu = cp.random.normal(size=(1000, 1000))
12 U_gpu, s_gpu, Vt_gpu = la.svd(A_gpu) # Ausfuehrung auf GPU

```

Listing 27: Array-API-kompatible SciPy-Funktion (Zukunftsvision)

SciPy hat mit Version 1.11 begonnen, Array-API-Kompatibilität in ausgewählten Submodulen (`scipy.special`, `scipy.fft`) zu implementieren, und plant die schrittweise Erweiterung auf alle Kernbereiche. Dies hat das Potenzial, SciPy zu einem vollständig hardwareagnostischen wissenschaftlichen Algorithmen-Stack zu machen.

7.1.7 Symbolisch-numerische Hybridmethoden

Die Integration symbolischer Mathematik (durch **SymPy**) mit numerischer Berechnung (durch NumPy und SciPy) hat sich zu einem eigenen Forschungsfeld entwickelt. **Lambdify** und **codegen** in SymPy überführen symbolisch hergeleitete Ausdrücke in numerisch auswertbaren Python-, NumPy- oder Fortran-Code:

```

1  from sympy import symbols, sin, diff, lambdify, Matrix
2  import numpy as np
3
4  x, t = symbols('x t')
5  omega = symbols('omega', positive=True)
6
7  # Symbolische Herleitung der Dispersionsrelation
8  u = sin(x - omega * t)
9  phase_velocity = omega / (x.diff(x)) # Symbolisch
10
11 # Jacobi-Matrix eines nichtlinearen Systems
12 F = Matrix([x**2 + t*x - 1, x*t - t**2])
13 J = F.jacobian([x, t])

```

```
14 print("Jacobi-Matrix:", J)
15
16 # Numerisch auswertbar via lambdify (NumPy-Backend)
17 jacobian_num = lambdify([x, t], J, modules='numpy')
18 J_values = jacobian_num(np.linspace(0, 1, 100), 0.5)
```

Listing 28: SymPy + NumPy: Symbolisch-numerische Pipeline

Zukünftige Entwicklungen werden die Symbiose von SymPy und SciPy vertiefen: Automatisch generierte Jacobimatrizen für SciPy-Optimierer, symbolisch hergeleitete Vorkonditionierer für lineare Gleichungssysteme und analytisch bekannte Integrationskerne für numerische Quadratur sind nur einige der vielversprechenden Anknüpfungspunkte.

7.1.8 Interaktive und webbasierte Visualisierung

Matplotlib hat seinen Ursprung in der Erzeugung statischer, druckbereiter Abbildungen. Die zunehmende Verlagerung wissenschaftlicher Arbeit in browserbasierte Umgebungen (Jupyter Lab, VS Code, Google Colab) und interaktive Dashboards hat eine neue Generation von Visualisierungswerkzeugen hervorgebracht, die auf den Konzepten von Matplotlib aufbauen oder mit ihr interagieren:

Plotly bietet deklarative, interaktive Grafiken mit Zoom, Hover-Tooltips und verknüpften Ansichten. Die Plotly-Express-API folgt eng der Matplotlib-Semantik und ermöglicht die Migration bestehender Matplotlib-Workflows.

Bokeh ist auf datengetriebene Webanwendungen und Streaming-Daten optimiert und ermöglicht die Einbettung wissenschaftlicher Visualisierungen in HTML-Seiten ohne serverseitige Infrastruktur.

HoloViews / HvPlot abstrahiert über verschiedene Visualisierungs-Backends (Matplotlib, Bokeh, Plotly) und erlaubt die deklarative Beschreibung von Datensichten, deren konkrete Darstellung vom gewählten Backend abhängt.

ipywidgets + Matplotlib ermöglichen vollständig interaktive Parameterstudien in Jupyter-Notebooks: Schieberegler, Dropdown-Menüs und Textfelder steuern direkt die Parameter von NumPy/SciPy-Berechnungen und aktualisieren die Matplotlib-Darstellung in Echtzeit.

WebAssembly / Pyodide transportiert den gesamten Scientific-Python-Stack – einschließlich NumPy, SciPy und Matplotlib – direkt in den Webbrowser. Bildungsinstitutionen berichten bereits von vollständig serverlos betriebenen interaktiven Lern-Umgebungen.

Matplotlib selbst antwortet auf diese Entwicklungen durch erweiterte Backend-Unterstützung (`ipympl/widget`-Backend für Jupyter) und ein zunehmend feingranulares Artist-Modell, das programmatische Animationen und interaktive Elemente erleichtert. Zu erwarten ist, dass Matplotlib als Rendering-Grundlage für höherschichtige interaktive Frameworks bestehen bleibt, während die direkte Nutzung für statische Publikationsgrafiken weiterhin unübertroffen ist.

7.1.9 Wissenschaftliches maschinelles Lernen und datengetriebene Modellentdeckung

Die Verbindung zwischen klassischen numerischen Methoden und modernem maschinellem Lernen manifestiert sich in einem rasch wachsenden Feld, das unter den Bezeichnungen *Scientific Machine Learning* (SciML) oder *AI for Science* bekannt ist. NumPy, SciPy und Matplotlib nehmen dabei eine Doppelrolle ein: als Werkzeuge zur Datenerzeugung und -vorverarbeitung einerseits, und als Referenzimplementierungen, gegen die lernbasierte Ansätze validiert werden, andererseits.

Konkrete Entwicklungsfelder umfassen:

- **Sparse Identification of Nonlinear Dynamics (SINDy)**: Datengetriebene Herleitung von Differentialgleichungen aus Zeitreihen. NumPy-basierte Bibliotheken wie PySINDy rekonstruieren aus Messdaten automatisch die dominanten Terme eines DGL-Systems – etwa die Lotka-Volterra-Gleichungen aus simulierten Populationsdaten, ohne a-priori-Kenntnis des Modells.
- **Operatorlernen (DeepONet, Fourier Neural Operator)**: Neuronale Netze, die anstelle von Funktionswerten ganze Lösungsoperatoren von PDEs lernen. SciPy-Löser erzeugen die Trainingsdaten; die gelernten Operatoren können anschließend Vorhersagen 10^4 -mal schneller als traditionelle Löser liefern.
- **Bayesianische Optimierung mit SciPy als Unterbau**: Bibliotheken wie `scikit-optimize` und `GPyOpt` verwenden `scipy.optimize` für interne Akquisitionsfunktions-Optimierungen und `scipy.stats` für die Beschreibung von Prioris.
- **Normalizing Flows und differenzierbare Simulationen**: NumPy-Daten treiben das Training von Wahrscheinlichkeitsdichte-Abschätzungsmodellen in der Astro- und Teilchenphysik.

7.1.10 Reproduzierbare Wissenschaft und FAIR-Datenprinzipien

Mit der wachsenden Veröffentlichungspflicht für Forschungsdaten und -code gewinnen Fragen der Reproduzierbarkeit wissenschaftlicher Ergebnisse an institutioneller Bedeutung [52, 53]. NumPy, SciPy und Matplotlib reagieren auf diese Herausforderung durch mehrere strukturelle Eigenschaften:

Semantische Versionierung und Deprecation-Policies : Alle drei Bibliotheken folgen strikten Versionierungsrichtlinien, die mehrjährige Rückwärtskompatibilität gewährleisten. Wissenschaftliche Paper, deren Berechnungen auf NumPy 1.x basieren, können durch `pip install numpy==1.26` reproduziert werden.

Zitierbarkeit : Mit den DOI-verlinkten Nature-Publikationen [1, 6, 8] sind alle drei Bibliotheken vollständig zitierfähig, ein wichtiger Meilenstein für ihre Anerkennung als Forschungsinfrastruktur.

Jupyter und nbformat : Das standardisierte Notebook-Format ermöglicht die vollständige Einbettung von NumPy/SciPy/Matplotlib-basierten Analysen in reproduzierbare, ausführbare Dokumente. Mit `nbconvert` und `papermill` können diese automatisiert ausgeführt werden.

FAIR-kompatible Datenformate : `scipy.io` liest und schreibt HDF5- und NetCDF-Dateien, die den FAIR-Datenprinzipien (Findable, Accessible, Interoperable, Reusable) genügen. Ergänzende Bibliotheken wie `zarr` und `h5py` bauen direkt auf NumPy-Arrays auf.

Zukünftige Entwicklungen werden die Integration mit dem verbreiteten Daten-Repository-Dienst **Zenodo** und dem Open-Science-Framework **OSF** stärken, sodass NumPy/SciPy-Analysen direkt aus dem Code heraus mit persistenten Identifikatoren archiviert werden können.

7.1.11 Hochleistungsrechnen und Exascale-Computing

Die Ära des *Exascale-Computing* – mit Systemen, die mehr als 10^{18} Gleitkommaoperationen pro Sekunde ausführen – stellt das wissenschaftliche Python-Ökosystem vor neuartige Herausforderungen. Auf Maschinen mit Hunderttausenden von CPU-Kernen und Tausenden von GPU-Knoten sind herkömmliche NumPy-Arrays inadequat. Dennoch bilden sie die konzeptuelle Grundlage für eine neue Generation von HPC-Abstraktionen:

- **mpi4py + NumPy**: Die Pythonbindung des Message Passing Interface (MPI) überträgt NumPy-Array-Segmente direkt zwischen Rechenknoten, ohne Datenkopien. NumPy-basierter Code läuft dadurch auf Supercomputern mit wenigen hundert Zeilen Boilerplate.
- **PETSc / SLEPc Python-Interfaces**: Über `petsc4py` sind die weltführenden Bibliotheken für skalierbare lineare Algebra und Eigenwertprobleme aus Python nutzbar – mit NumPy-Arrays als Daten-Übergabeschnittstelle.
- **FEniCSx und Firedrake**: Finite-Elemente-Frameworks für großskalige PDE-Simulationen integrieren NumPy/SciPy als Postprocessing- und Visualisierungs-Backend über `pyvista` und `Matplotlib`.
- **PyFR und OpenFOAM Python-Bindings**: Strömungsmechanik- Codes der industriellen und akademischen Spitzenforschung bieten zunehmend Python-Schnittstellen, die NumPy-Arrays als primäres Datenformat verwenden.

Langfristig wird erwartet, dass SciPy dedizierte Parallelversionen seiner Kernalgorithmen einführt – verteilte FFTs (basierend auf `pfft` oder `mpiFFT4py`), parallele Gleichungslöser und skalierbare Optimierungsroutinen.

7.1.12 Quantencomputing und Quanten-Simulation

Das Aufkommen von Quantencomputern mit 100–1 000 physischen Qubits eröffnet völlig neue Möglichkeiten für die wissenschaftliche Berechnung. NumPy und SciPy spielen in diesem Kontext eine doppelte Rolle:

Einerseits sind sie die primären Werkzeuge für die *klassische Simulation von Quantensystemen*. Bibliotheken wie **QuTiP** (Quantum Toolbox in Python) verwenden dichte NumPy-Matrizen und SciPy-Eigenwertlöser, um Hamiltonoperatoren, Dichtematrizen und Lindbladgleichungen für offene Quantensysteme zu berechnen:

```

1 import qutip as qt
2 import numpy as np
3
4 # Zwei-Niveau-System (Qubit)
5 H = qt.sigmax()          # Hamiltonoperator: sigma_x
6 psi0 = qt.basis(2, 0)    # Anfangszustand |0>
7
8 tlist = np.linspace(0, 10, 200) # Zeitvektor (NumPy)
9 # Schroedinger-Gleichung loesen (intern: SciPy solve_ivp)
10 result = qt.mesolve(H, psi0, tlist, [], [qt.sigmax(), qt.sigmaz()])
11
12 # Erwartungswerte visualisieren (Matplotlib)
13 import matplotlib.pyplot as plt
14 plt.plot(tlist, result.expect[0], label=r'$\langle\sigma_x\rangle$')
15 plt.plot(tlist, result.expect[1], label=r'$\langle\sigma_z\rangle$')

```

Listing 29: QuTiP: Quantendynamik mit NumPy/SciPy

Andererseits bieten **Qiskit** (IBM), **Cirq** (Google) und **PennyLane** Python-Schnittstellen für echte Quanten-Hardware, die intern NumPy für die Darstellung von Quantenzustandsvektoren und Messstatistiken nutzen. Mit der Konsolidierung der Quantenhardware wird NumPy als gemeinsames Datenformat die Brücke zwischen klassischer und Quantenberechnung bilden.

7.1.13 Nachhaltigkeit und Community-Governance des Ökosystems

Eine oft unterschätzte Dimension der Zukunftsperspektiven betrifft nicht die technologische, sondern die soziale und institutionelle Nachhaltigkeit des Scientific-Python-Ökosystems. Die vergangenen Jahre haben gezeigt, dass hochwertige Open-Source-Infrastruktur für die Wissenschaft dauerhafter Finanzierung und stabiler Governance bedarf:

NumFOCUS : Die gemeinnützige Organisation finanziert und koordiniert die Entwicklung von NumPy, SciPy, Matplotlib und über 40 weiteren wissenschaftlichen Python-Projekten durch Förderanträge, Konferenzen (SciPy Conference, PyData) und Unternehmenssponsoring.

SPEC-Prozess (Scientific Python Ecosystem Coordination) : Die *Scientific Python Ecosystem Coordination*-Dokumente standardisieren Versionierungsrichtlinien, Python-Mindestversionen und Interoperabilitätsnormen für alle Mitgliedsprojekte, um den enormen Koordinationsaufwand eines verteilten Ökosystems zu reduzieren.

Förderinstitutionen : Der Chan Zuckerberg Initiative *Essential Open Source for Science*-Fonds und das Alfred P. Sloan Foundation *Scientific Software*-Programm finanzieren Vollzeit- Entwicklerstellen für NumPy, SciPy und Matplotlib.

Diversität und Inklusion : Gezielte Programme wie *GSoC* (Google Summer of Code) und *Outreachy* erweitern den Entwicklerpool und stellen sicher, dass das Ökosystem die Anforderungen einer global diverse wissenschaftlichen Gemeinschaft widerspiegelt.

Die langfristige Perspektive ist optimistisch: Das dreifache Fundament aus technischer Exzellenz, institutioneller Verankerung und einer wachsenden, engagierten Community

garantiert, dass NumPy, SciPy und Matplotlib nicht nur kurzfristige Werkzeuge, sondern dauerhafter Bestandteil der wissenschaftlichen Infrastruktur bleiben werden.

7.1.14 Synthese: Eine unverzichtbare und zukunftssichere Plattform

Die vorstehenden Ausblicke auf GPU-Beschleunigung, differenzierbares Programmieren, maschinelles Lernen, Quantencomputing und Exascale-HPC verdeutlichen eine übergeordnete Erkenntnis: Alle diese Entwicklungslinien bauen auf denselben konzeptuellen Grundlagen auf, die NumPy, SciPy und Matplotlib vor mehr als zwei Jahrzehnten etabliert haben – das N-dimensionale Array als universelle Datenstruktur, hochoptimierte Algorithmen aus der numerischen Mathematik und publikationsreife Visualisierung als kommunikatives Endprodukt wissenschaftlicher Arbeit.

Jede neue Bibliothek – ob JAX, CuPy, Dask oder PyTorch – definiert ihre Schnittstellen in exakter Analogie zu NumPy, jeder neue Algorithmen-Stack orientiert sich an SciPy, und jede visualisierungsorientierte Anwendung erbt Designentscheidungen von Matplotlib. Dies ist keine Zufälligkeit, sondern Ausdruck tief verwurzelter Qualität: Diese drei Bibliotheken haben durch jahrzehntelangen Einsatz in allen Gebieten der Wissenschaft bewiesen, dass ihre Abstraktionen die richtige Granularität besitzen, um sowohl universell als auch performant zu sein.

Das Scientific-Python-Ökosystem steht nicht am Ende einer Entwicklung, sondern am Beginn einer neuen Phase, in der die bewährten Prinzipien von NumPy, SciPy und Matplotlib auf heterogene Hardware, differenzierbares Rechnen, künstliche Intelligenz und planetare Datenskalen ausgedehnt werden. Wer heute NumPy, SciPy und Matplotlib beherrscht, hält den Generalschlüssel zu diesem gesamten zukünftigen Ökosystem in Händen.

Literatur

- [1] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.* Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. <https://doi.org/10.1038/s41586-020-2649-2>
- [2] T. E. Oliphant. *A Guide to NumPy*. Trelgol Publishing, 2006. <https://numpy.org/doc/stable/>
- [3] T. E. Oliphant. Python for Scientific Computing. *Computing in Science & Engineering*, 9(3):10–20, 2007. <https://doi.org/10.1109/MCSE.2007.58>
- [4] J. VanderPlas. *Python Data Science Handbook*. O’Reilly Media, 2016. <https://jakevdp.github.io/PythonDataScienceHandbook/>
- [5] W. McKinney. *Python for Data Analysis*, 2. Auflage. O’Reilly Media, 2017.
- [6] P. Virtanen, R. Gommers, T. E. Oliphant *et al.* SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020. <https://doi.org/10.1038/s41592-019-0686-2>
- [7] E. Jones, T. E. Oliphant, P. Peterson *et al.* SciPy: Open Source Scientific Tools for Python, 2001. <https://www.scipy.org/>
- [8] J. D. Hunter. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. <https://doi.org/10.1109/MCSE.2007.55>

- [9] K. Dale, S. Murray und V. Tolochko. *Data Visualization with Python and JavaScript*. O'Reilly Media, 2015.
- [10] A. Devert. *matplotlib Plotting Cookbook*. Packt Publishing, 2014.
- [11] R. Johansson. *Numerical Python: Scientific Computing and Data Science Applications with NumPy, SciPy and Matplotlib*, 2. Auflage. Apress, 2019. <https://doi.org/10.1007/978-1-4842-4246-4>
- [12] H. P. Langtangen. *A Primer on Scientific Programming with Python*, 5. Auflage. Springer, 2016. <https://doi.org/10.1007/978-3-662-49887-3>
- [13] A. Scopatz und K. D. Huff. *Effective Computation in Physics*. O'Reilly Media, 2015.
- [14] W. H. Press, S. A. Teukolsky, W. T. Vetterling und B. P. Flannery. *Numerical Recipes: The Art of Scientific Computing*, 3. Auflage. Cambridge University Press, 2007.
- [15] G. H. Golub und C. F. van Loan. *Matrix Computations*, 4. Auflage. Johns Hopkins University Press, 2013.
- [16] L. N. Trefethen. *Spectral Methods in MATLAB*. SIAM, 2000. <https://doi.org/10.1137/1.9780898719598>
- [17] R. L. Burden, J. D. Faires und A. M. Burden. *Numerical Analysis*, 10. Auflage. Cengage Learning, 2015.
- [18] K. E. Atkinson. *An Introduction to Numerical Analysis*, 2. Auflage. John Wiley & Sons, 1989.
- [19] A. V. Oppenheim und R. W. Schaffer. *Discrete-Time Signal Processing*, 3. Auflage. Prentice Hall, 2010.
- [20] R. G. Lyons. *Understanding Digital Signal Processing*, 3. Auflage. Prentice Hall, 2011.
- [21] J. W. Cooley und J. W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965. <https://doi.org/10.1090/S0025-5718-1965-0178586-1>
- [22] T. Hastie, R. Tibshirani und J. Friedman. *The Elements of Statistical Learning*, 2. Auflage. Springer, 2009. <https://doi.org/10.1007/978-0-387-84858-7>
- [23] C. M. Grinstead und J. L. Snell. *Introduction to Probability*, 2. Auflage. American Mathematical Society, 2012. <https://math.dartmouth.edu/~prob/prob/prob.pdf>
- [24] M. H. DeGroot und M. J. Schervish. *Probability and Statistics*, 4. Auflage. Pearson, 2012.
- [25] A. J. Lotka. *Elements of Physical Biology*. Williams and Wilkins, 1925.
- [26] V. Volterra. Variazioni e fluttuazioni del numero d'individui in specie animali conviventi. *Mem. Accad. Lincei Roma*, 2:31–113, 1926.
- [27] E. N. Lorenz. Deterministic Nonperiodic Flow. *Journal of the Atmospheric Sciences*, 20(2):130–141, 1963. [https://doi.org/10.1175/1520-0469\(1963\)020<0130:DNF>2.0.CO;2](https://doi.org/10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2)

- [28] S. H. Strogatz. *Nonlinear Dynamics and Chaos*, 2. Auflage. Westview Press, 2018.
- [29] E. Hairer, S. P. Nørsett und G. Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2. Auflage. Springer, 1993. <https://doi.org/10.1007/978-3-540-78862-1>
- [30] E. Hairer und G. Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, 2. Auflage. Springer, 1996. <https://doi.org/10.1007/978-3-642-05221-7>
- [31] M. Abramowitz und I. A. Stegun. *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. Dover Publications, 1972. <https://personal.math.ubc.ca/~cbm/aands/>
- [32] F. W. J. Olver, D. W. Lozier, R. F. Boisvert und C. W. Clark (Hrsg.). *NIST Handbook of Mathematical Functions*. Cambridge University Press, 2010. <https://dlmf.nist.gov/>
- [33] J. Nocedal und S. J. Wright. *Numerical Optimization*, 2. Auflage. Springer, 2006. <https://doi.org/10.1007/978-0-387-40065-5>
- [34] S. Boyd und L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. <https://doi.org/10.1017/CB09780511804441>
- [35] L. N. Trefethen und D. Bau III. *Numerical Linear Algebra*. SIAM, 1997. <https://doi.org/10.1137/1.9780898719574>
- [36] R. A. Horn und C. R. Johnson. *Matrix Analysis*, 2. Auflage. Cambridge University Press, 2012.
- [37] F. Pérez und B. E. Granger. IPython: A System for Interactive Scientific Computing. *Computing in Science & Engineering*, 9(3):21–29, 2007. <https://doi.org/10.1109/MCSE.2007.53>
- [38] T. Kluyver, B. Ragan-Kelley, F. Pérez *et al.* Jupyter Notebooks – a publishing format for reproducible computational workflows. In: F. Loizides und B. Schmidt (Hrsg.), *Positioning and Power in Academic Publishing*. IOS Press, S. 87–90, 2016. <https://doi.org/10.3233/978-1-61499-649-1-87>
- [39] F. Pedregosa, G. Varoquaux, A. Gramfort *et al.* Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. <https://jmlr.org/papers/v12/pedregosa11a.html>
- [40] The Pandas Development Team. pandas-dev/pandas: Pandas. *Zenodo*, 2020. <https://doi.org/10.5281/zenodo.3509134>
- [41] J. Bradbury, R. Frostig, P. Hawkins *et al.* JAX: composable transformations of Python+NumPy programs, 2018. <https://github.com/google/jax>
- [42] R. Okuta, Y. Unno, D. Nishino, S. Hido und C. Loomis. CuPy: A NumPy-Compatible Matrix Library Accelerated by CUDA. In: *Workshop Proceedings of NeurIPS 2017*. <https://cupy.dev/>

- [43] B. P. Abbott *et al.*, LIGO Scientific Collaboration und Virgo Collaboration. Observation of Gravitational Waves from a Binary Black Hole Merger. *Physical Review Letters*, 116(6):061102, 2016. <https://doi.org/10.1103/PhysRevLett.116.061102>
- [44] Event Horizon Telescope Collaboration *et al.* First M87 Event Horizon Telescope Results. I. The Shadow of the Supermassive Black Hole. *The Astrophysical Journal Letters*, 875(1):L1, 2019. <https://doi.org/10.3847/2041-8213/ab0ec7>
- [45] Astropy Collaboration *et al.* Astropy: A community Python package for astronomy. *Astronomy & Astrophysics*, 558:A33, 2013. <https://doi.org/10.1051/0004-6361/201322068>
- [46] P. Dierckx. *Curve and Surface Fitting with Splines*. Oxford University Press, 1993.
- [47] G. E. Fasshauer. *Meshfree Approximation Methods with MATLAB*. World Scientific, 2007. <https://doi.org/10.1142/6437>
- [48] P. Kovesi. Good Colour Maps: How to Design Them. *arXiv preprint arXiv:1509.03700*, 2015. <https://arxiv.org/abs/1509.03700>
- [49] Y. Liu und J. Heer. Somewhere Over the Rainbow: An Empirical Assessment of Quantitative Colormaps. In: *Proceedings of ACM CHI 2018*. <https://doi.org/10.1145/3173574.3174172>
- [50] M. Lutz. *Learning Python*, 5. Auflage. O'Reilly Media, 2013.
- [51] A. Martelli. *Python in a Nutshell*, 2. Auflage. O'Reilly Media, 2006.
- [52] D. L. Donoho. An invitation to reproducible computational research. *Biostatistics*, 11(3):385–388, 2010. <https://doi.org/10.1093/biostatistics/kxq028>
- [53] R. D. Peng. Reproducible Research in Computational Science. *Science*, 334(6060):1226–1227, 2011. <https://doi.org/10.1126/science.1213847>
- [54] NumPy Development Team. *NumPy Reference Documentation*, 2024. <https://numpy.org/doc/stable/reference/>
- [55] SciPy Development Team. *SciPy Reference Documentation*, 2024. <https://docs.scipy.org/doc/scipy/>
- [56] Matplotlib Development Team. *Matplotlib Documentation*, 2024. <https://matplotlib.org/stable/>
- [57] NumPy Developers. *NumPy Source Code Repository*, 2024. <https://github.com/numpy/numpy>
- [58] SciPy Developers. *SciPy Source Code Repository*, 2024. <https://github.com/scipy/scipy>
- [59] Matplotlib Developers. *Matplotlib Source Code Repository*, 2024. <https://github.com/matplotlib/matplotlib>